

Deep Learning: Theory and Practice

Recurrent Neural Networks

30-04-2019

Introduction

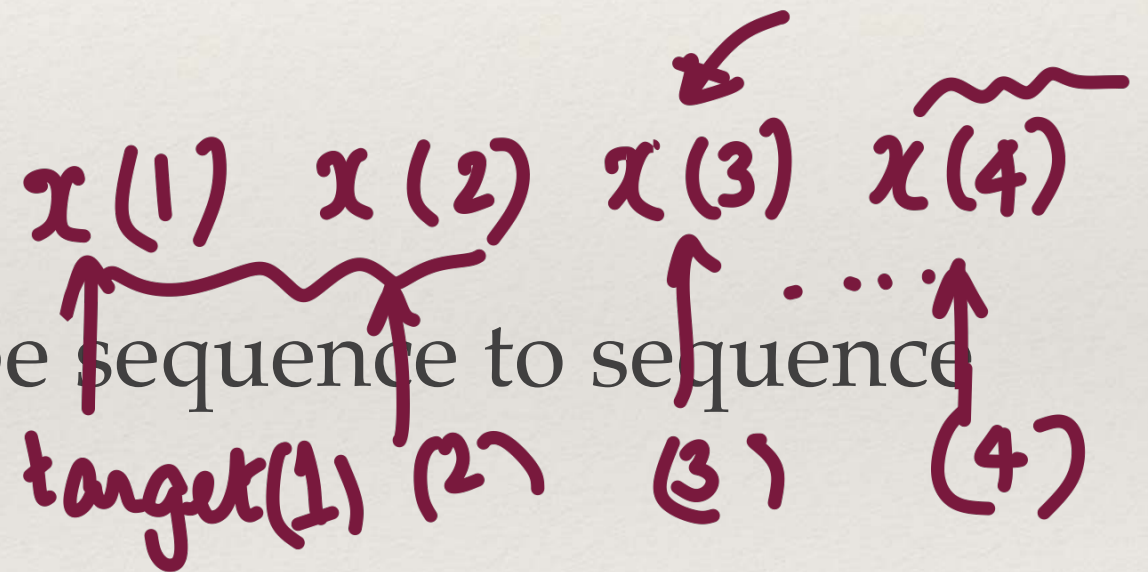
- ❖ The standard DNN / CNN paradigms

- ❖ $(\underline{x}, \underline{y})$ - ordered pair of data vectors / images ($\underline{x}$) and target ($\underline{y}$)

- ❖ Moving to sequence data

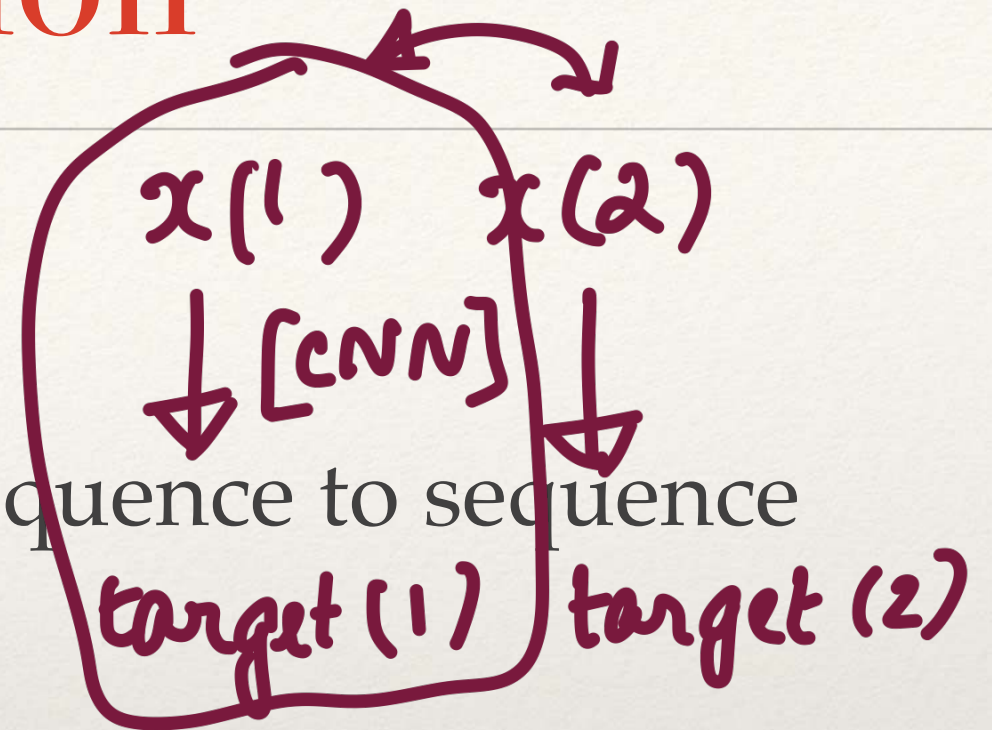
- ❖ $(x(t), y(t))$ where this could be sequence to sequence mapping task.

- ❖ $(x(t), \mathbf{y})$ where this could be a sequence to vector mapping task.



Introduction

- ❖ Difference between CNNs / DNNs
- ❖ $(x(t), y(t))$ where this could be sequence to sequence mapping task.
- ❖ Input features / output targets are correlated in time.
- ❖ Unlike standard models where each pair is independent.
- ❖ Need to model dependencies in the sequence over time.



Chap 10 of Deep Learning Book Introduction to Recurrent Networks

Recurrence

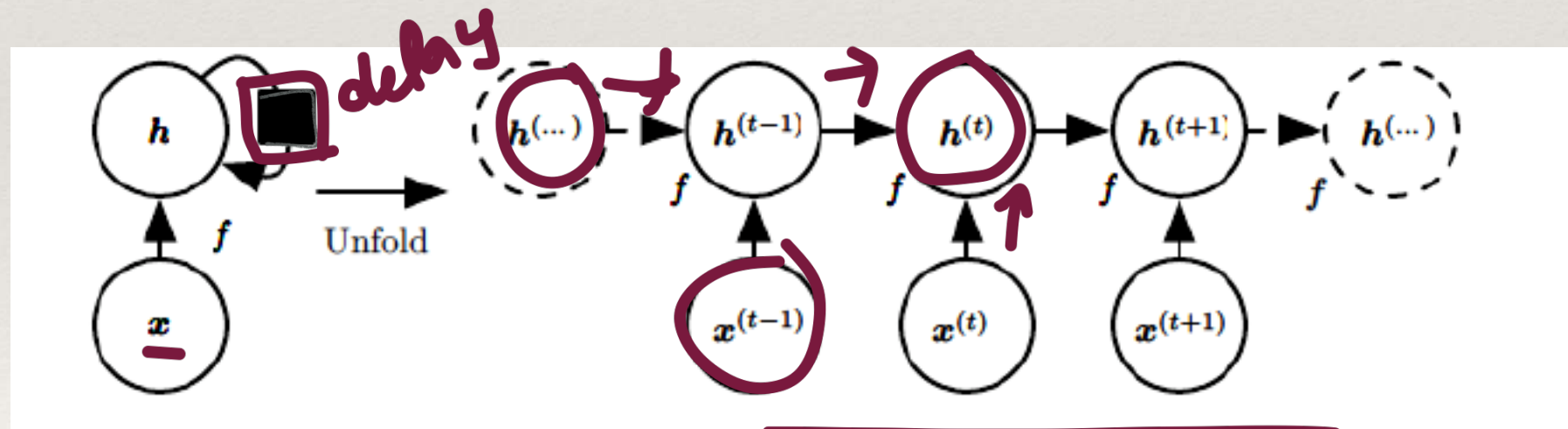
$$s^{(t)} = f(s^{(t-1)}; \theta),$$

$$\begin{aligned} s^{(3)} &= f(s^{(2)}; \theta) \\ &= f(f(s^{(1)}; \theta); \theta) \end{aligned}$$

$$s^{(t)} = f(s^{(t-1)}, x^{(t)}; \theta),$$

$$h^{(t)} = f(h^{(t-1)}, x^{(t)}; \theta),$$

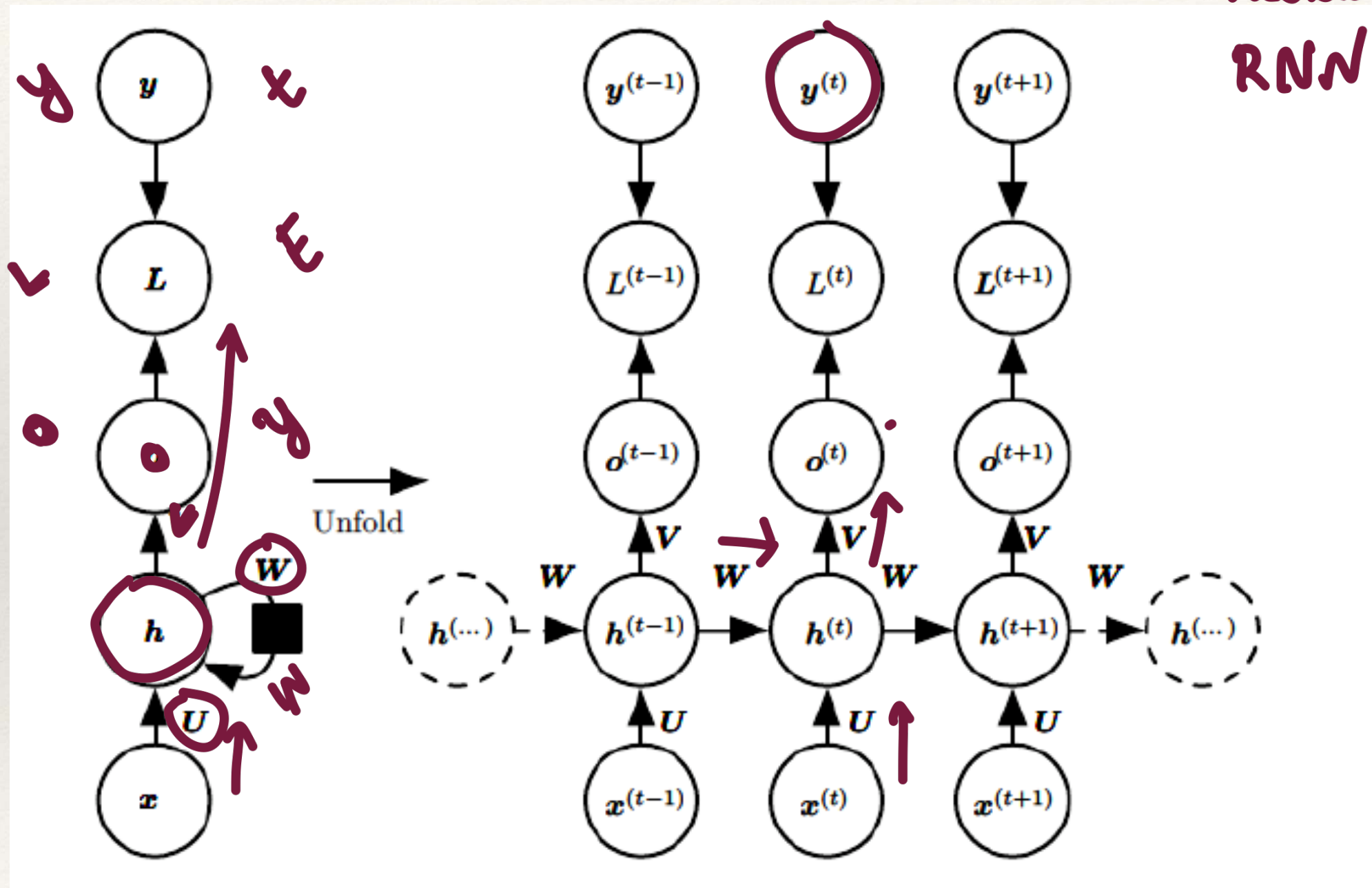
Neural network



Recurrent Networks

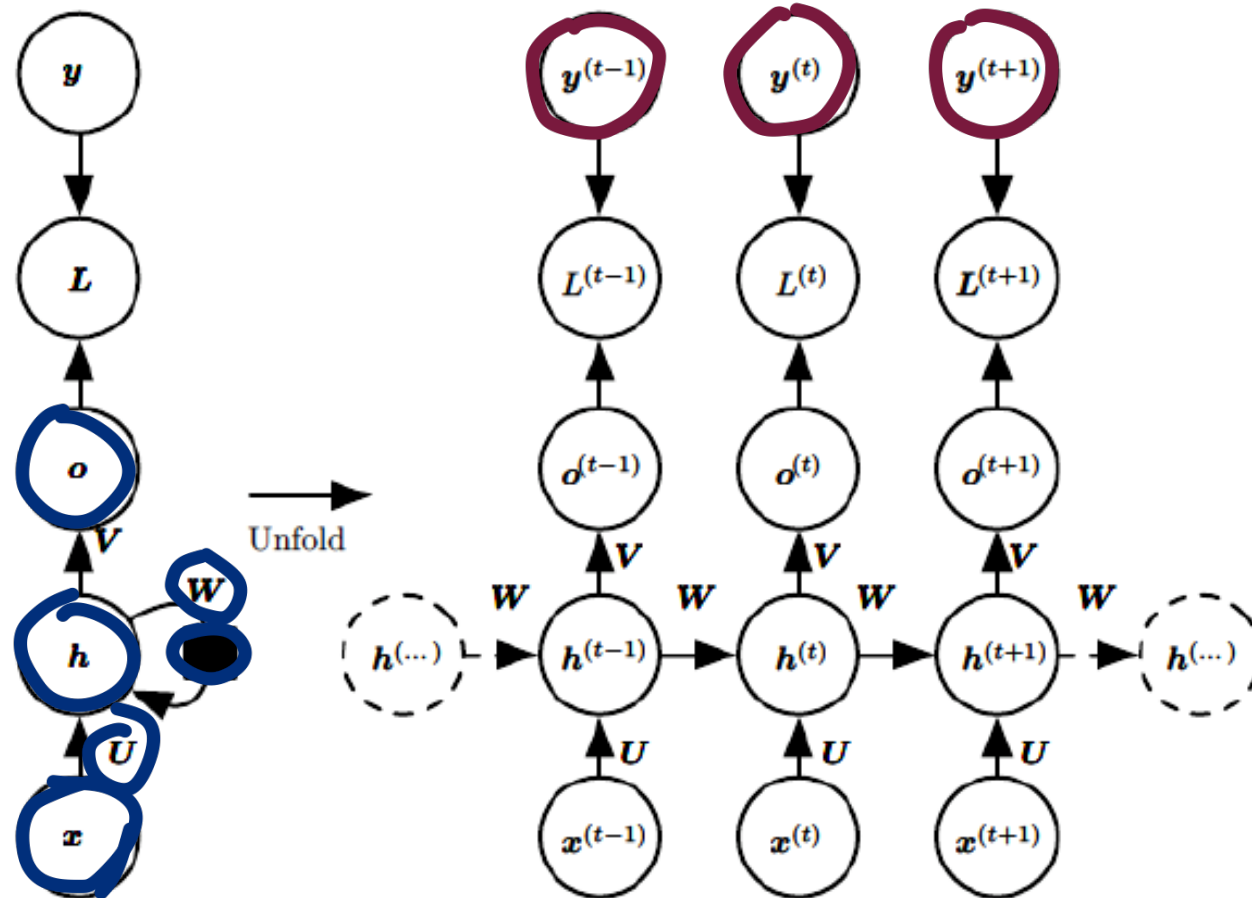
1-hidden layer

RNN



Recurrent Networks

$x_1 = \{x_1^{(1)} \dots x_1^{(\tau)}\}$ ← time
 $x_2 = \{x_2^{(1)} \dots x_2^{(\tau)}\}$ ← sequence index



$$\begin{aligned}
 \underline{a^{(t)}} &= \underline{b} + \underline{W} \underline{h^{(t-1)}} + \underline{U} \underline{x^{(t)}} \\
 \underline{h^{(t)}} &= \underline{\tanh(a^{(t)})} \\
 \underline{o^{(t)}} &= \underline{c + V h^{(t)}} \\
 \underline{\hat{y}^{(t)}} &= \underline{\text{softmax}(o^{(t)})}
 \end{aligned}$$

$\mathcal{L}(\hat{y}^{(t)}, y^{(t)})$

$$\begin{aligned}
 &L(\{x^{(1)} \dots x^{(\tau)}\}, \{y^{(1)}, \dots, y^{(\tau)}\}) \\
 &= \sum_t L^{(t)} \\
 &= - \sum_t \log p_{\text{model}}(y^{(t)} | \{x^{(1)}, \dots, x^{(t)}\})
 \end{aligned}$$

Back Propagation in RNNs

$$\begin{aligned}
 a^{(t)} &= b + W h^{(t-1)} + U x^{(t)} \\
 h^{(t)} &= \tanh(a^{(t)}) \\
 o^{(t)} &= c + V h^{(t)} \\
 \hat{y}^{(t)} &= \text{softmax}(o^{(t)})
 \end{aligned}$$

Handwritten notes: $(y^{(t)}, \hat{y}^{(t)})$ with arrows pointing to $y^{(t)}$ and $\hat{y}^{(t)}$ in the equations above.

$$\begin{aligned}
 &[x_1^{(1)}, x_2^{(1)}] \\
 &[x_1^{(2)}, x_2^{(2)}] \\
 &\vdots
 \end{aligned}$$

... $x^{(30)}$ April May

Model Parameters

$$\underline{U}, \underline{V}, \underline{W}, \underline{b} \text{ and } \underline{c}$$

Gradient Descent

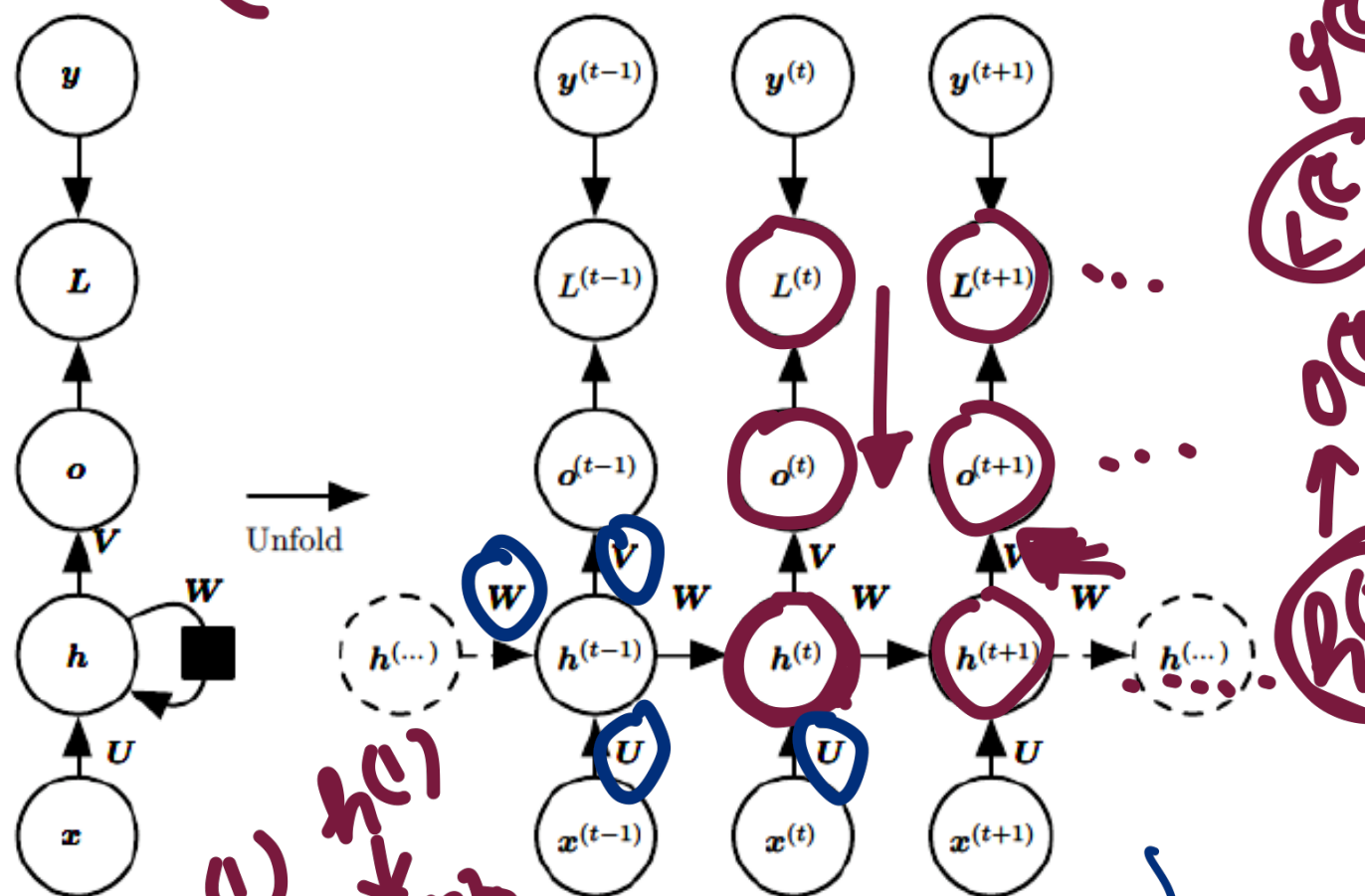
$$\begin{aligned}
 &L(\{x^{(1)}, \dots, x^{(\tau)}\}, \{y^{(1)}, \dots, y^{(\tau)}\}) \\
 &= \sum_t L^{(t)} \\
 &= - \sum_t \log p_{\text{model}}(y^{(t)} | \{x^{(1)}, \dots, x^{(t)}\})
 \end{aligned}$$

$$\frac{\partial L}{\partial L^{(t)}} = 1.$$

$$(\nabla_{o^{(t)}} L)_i = \frac{\partial L}{\partial o_i^{(t)}} = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}} = \hat{y}_i^{(t)} - \mathbf{1}_{i, y^{(t)}}$$

Recurrent Networks

(2)



$$h^{(t)} \rightarrow h^{(t)} + \epsilon \quad \lambda = y^{(t)}$$

$$\nabla_{o^{(t)} L}_i = \frac{\partial L}{\partial o_i^{(t)}} = \frac{\partial L}{\partial L^{(t)}} \frac{\partial L^{(t)}}{\partial o_i^{(t)}} = \hat{y}_i^{(t)} - \underline{\underline{1_{i, y^{(t)}}}}}$$

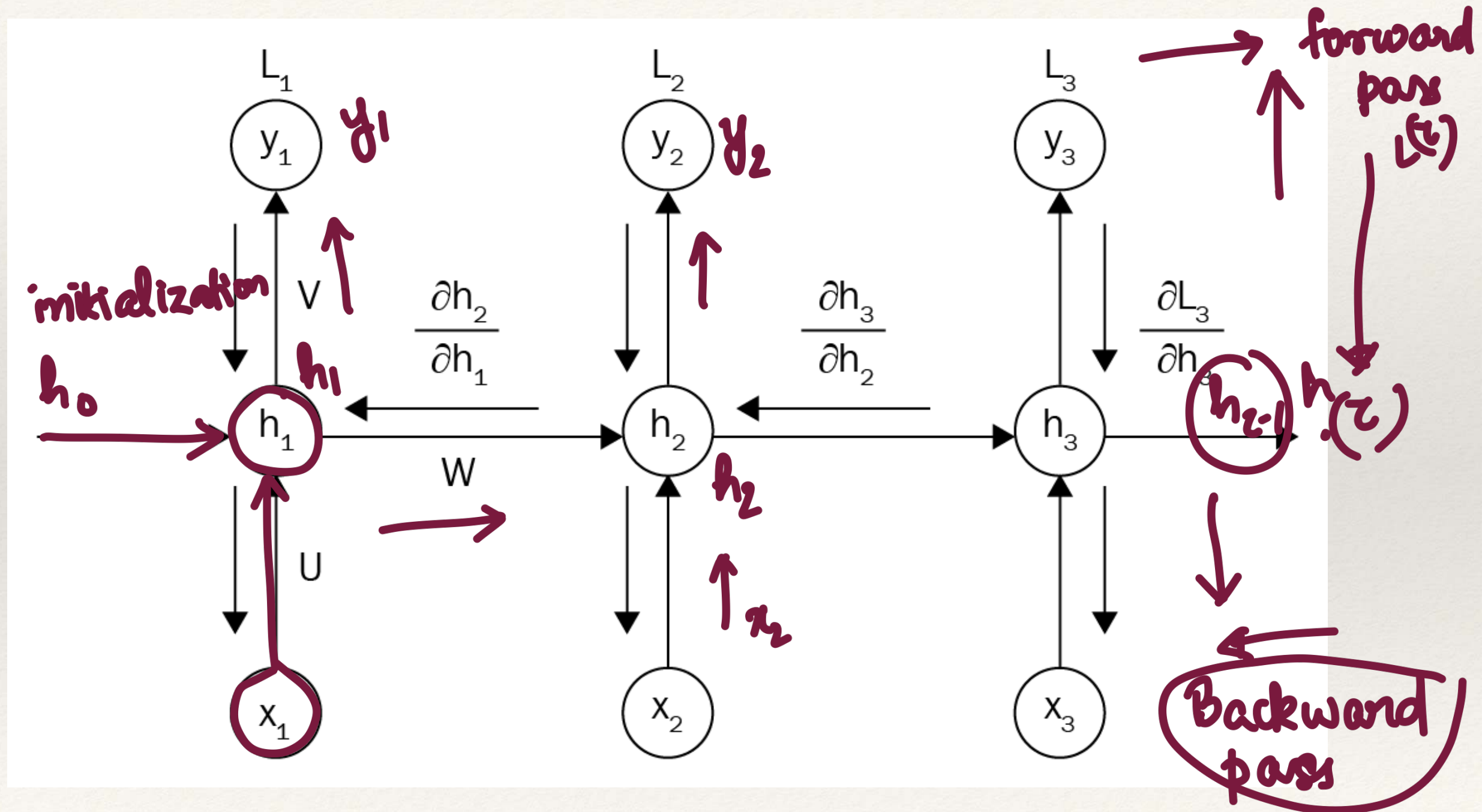
$$\nabla_{h^{(\tau)} L} = V^\top \nabla_{o^{(\tau)} L}$$

Backward Recurrence

$$\nabla_{h^{(t)} L} = \left(\frac{\partial h^{(t+1)}}{\partial h^{(t)}} \right)^\top (\nabla_{h^{(t+1)} L}) + \left(\frac{\partial o^{(t)}}{\partial h^{(t)}} \right)^\top \nabla_{o^{(t)} L}$$

$$= W^\top (\nabla_{h^{(t+1)} L}) \text{diag} \left(1 - \left(h^{(t+1)} \right)^2 \right) + V^\top (\nabla_{o^{(t)} L})$$

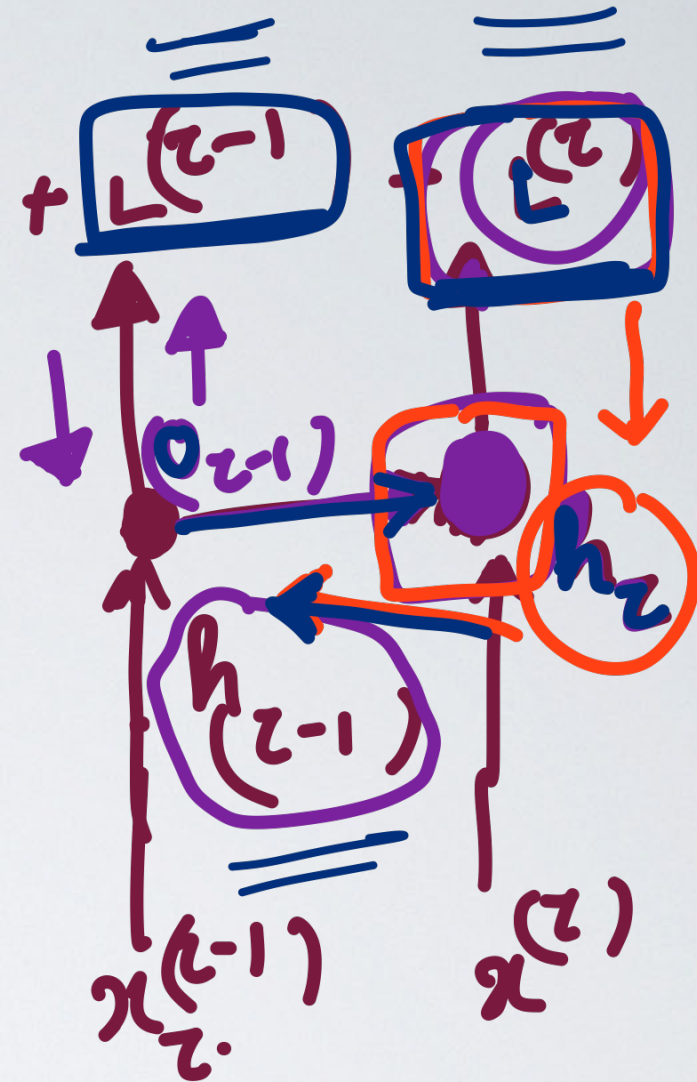
(BPTT) Back Propagation Through Time



$$\left[\frac{\partial L}{\partial h^{(t-1)}} \right]$$

$$L = \dots + L^{(t-1)} + L^{(t)}$$

$$h_{t-1} + \epsilon$$



$$o_t = v^T h_t + c$$

$$\hat{y}_t = \text{softmax}(o_t)$$

$$L^t = \text{CE}[y_t, \hat{y}_t]$$

$$\frac{\partial L}{\partial h^t} = v^T \left(\frac{\partial L}{\partial o_t} \right)$$

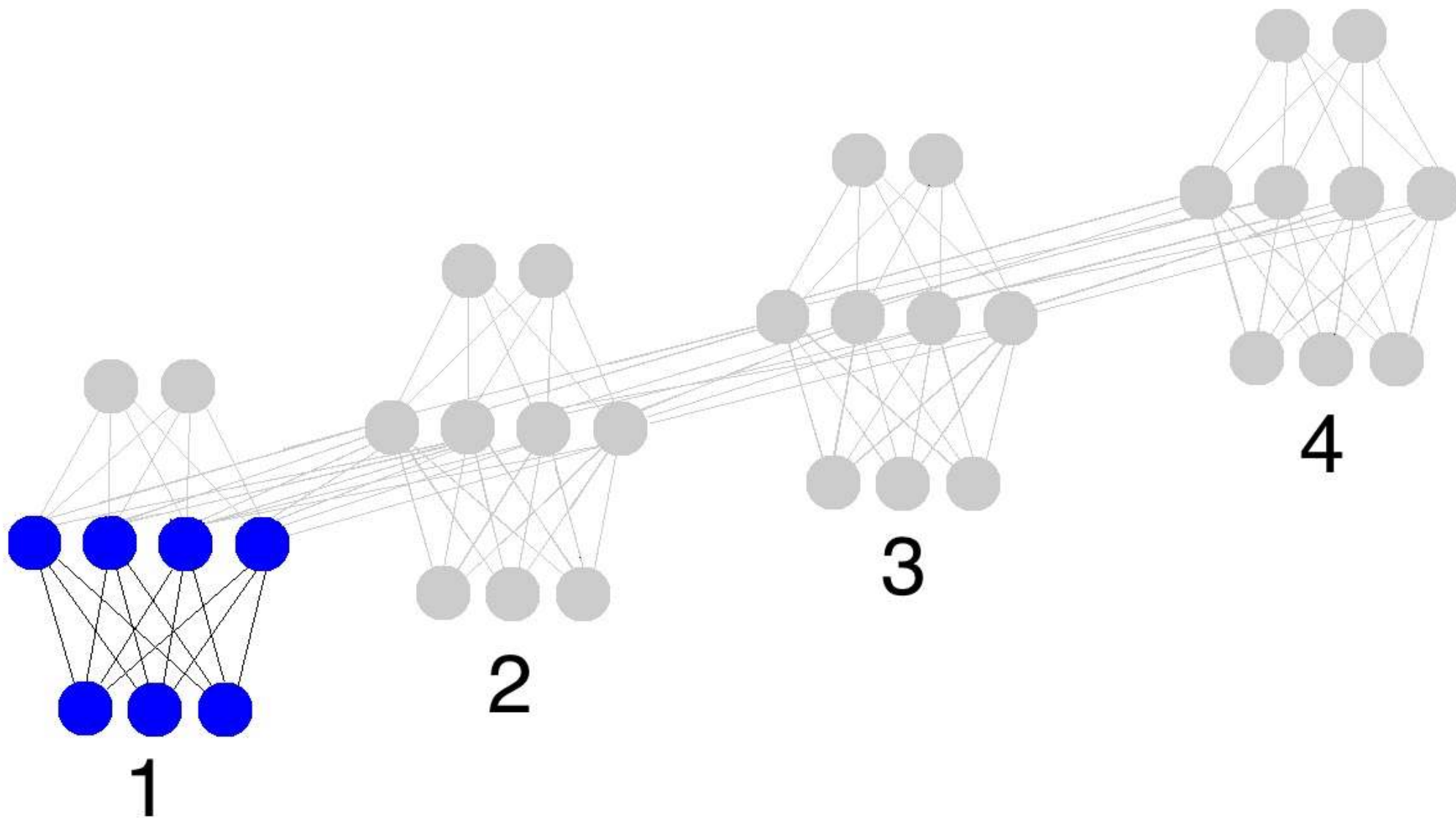
$$\frac{\partial L}{\partial h^{(t-1)}}$$

$$= v^T \frac{\partial L}{\partial o_{t-1}} + w^T (\text{der}_{\tanh}) \frac{\partial L}{\partial h^t}$$

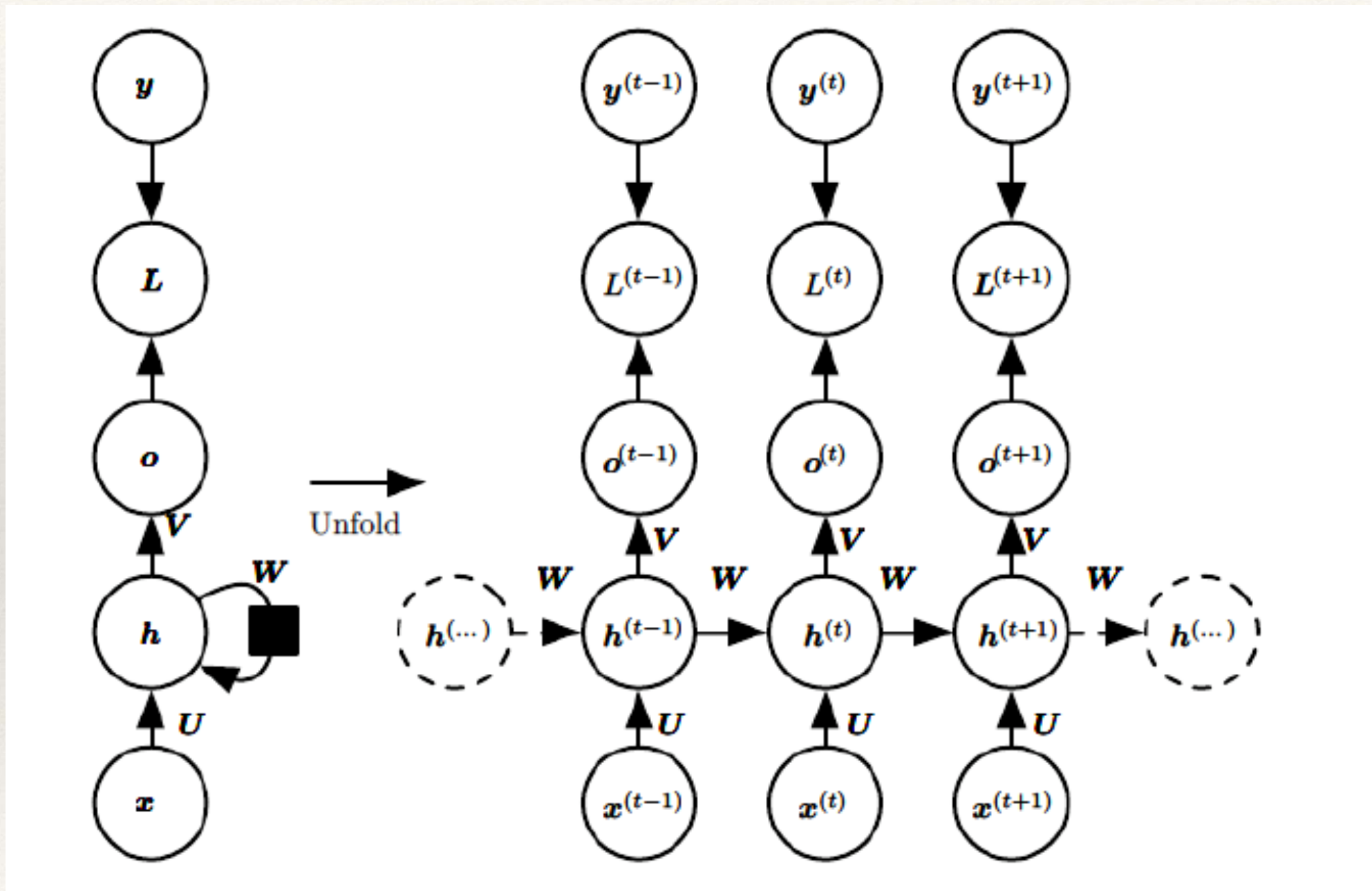
$$h_t = \tanh(W h^{(t-1)} + U x^{(t)} + b)$$



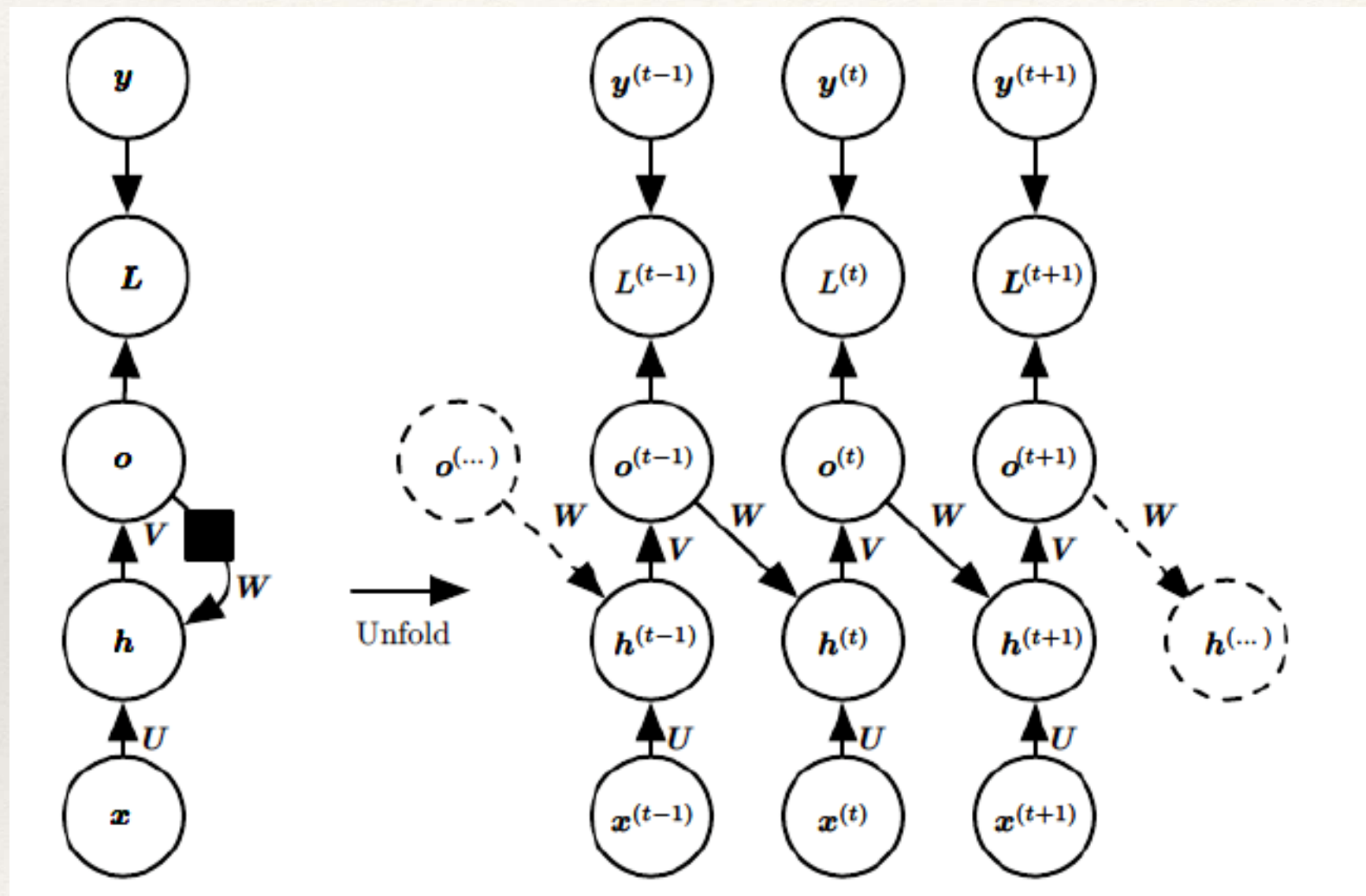
Back Propagation Through Time



Standard Recurrent Networks

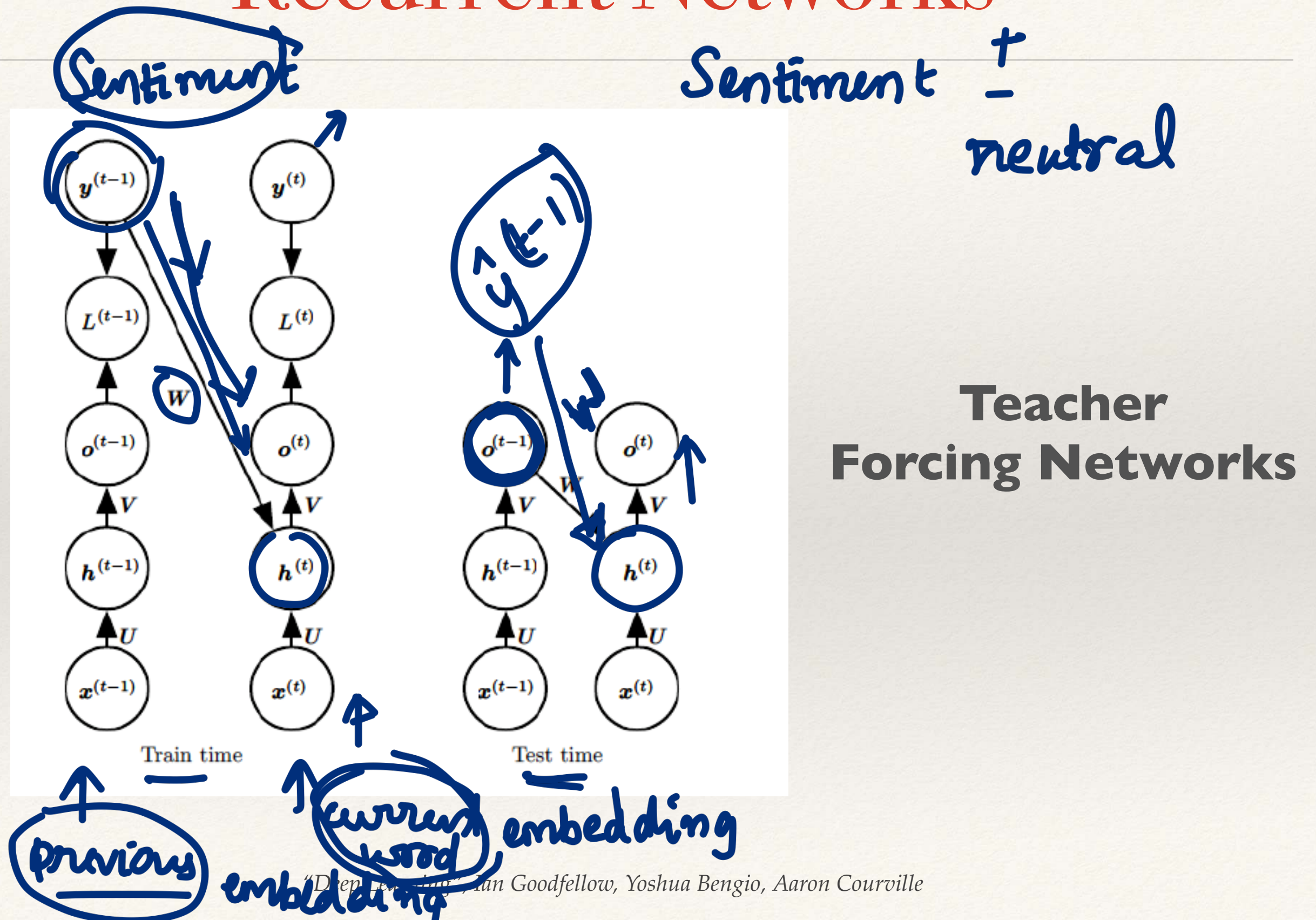


Other Recurrent Networks

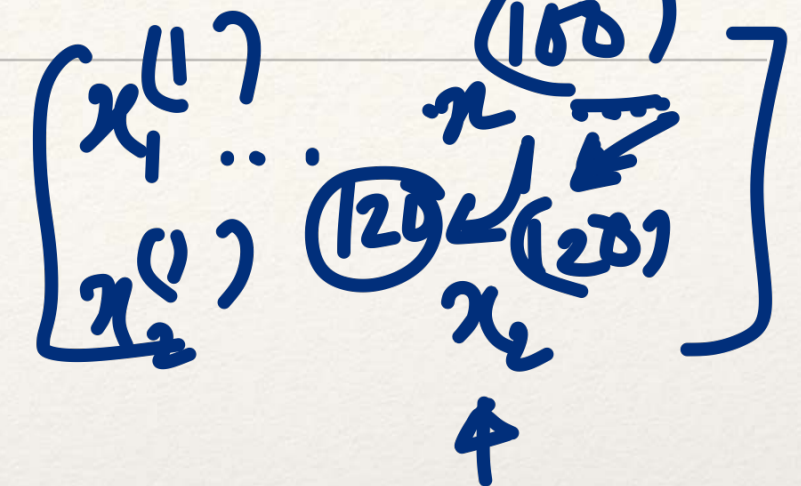
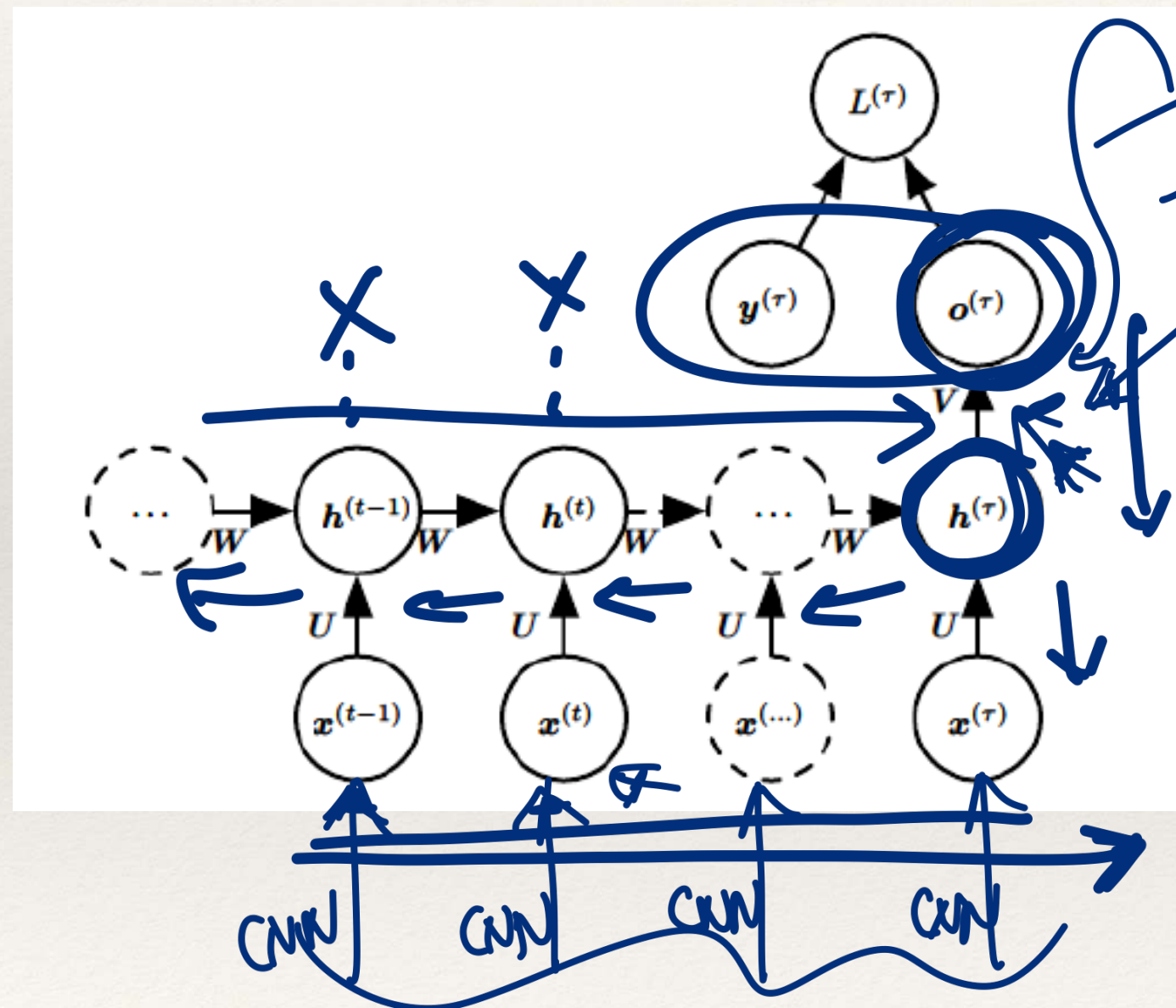


**Teacher
Forcing Networks**

Recurrent Networks



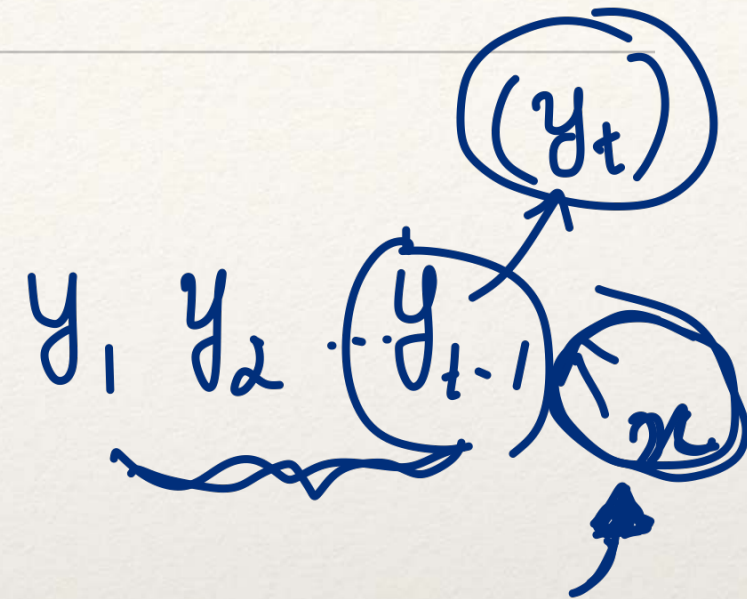
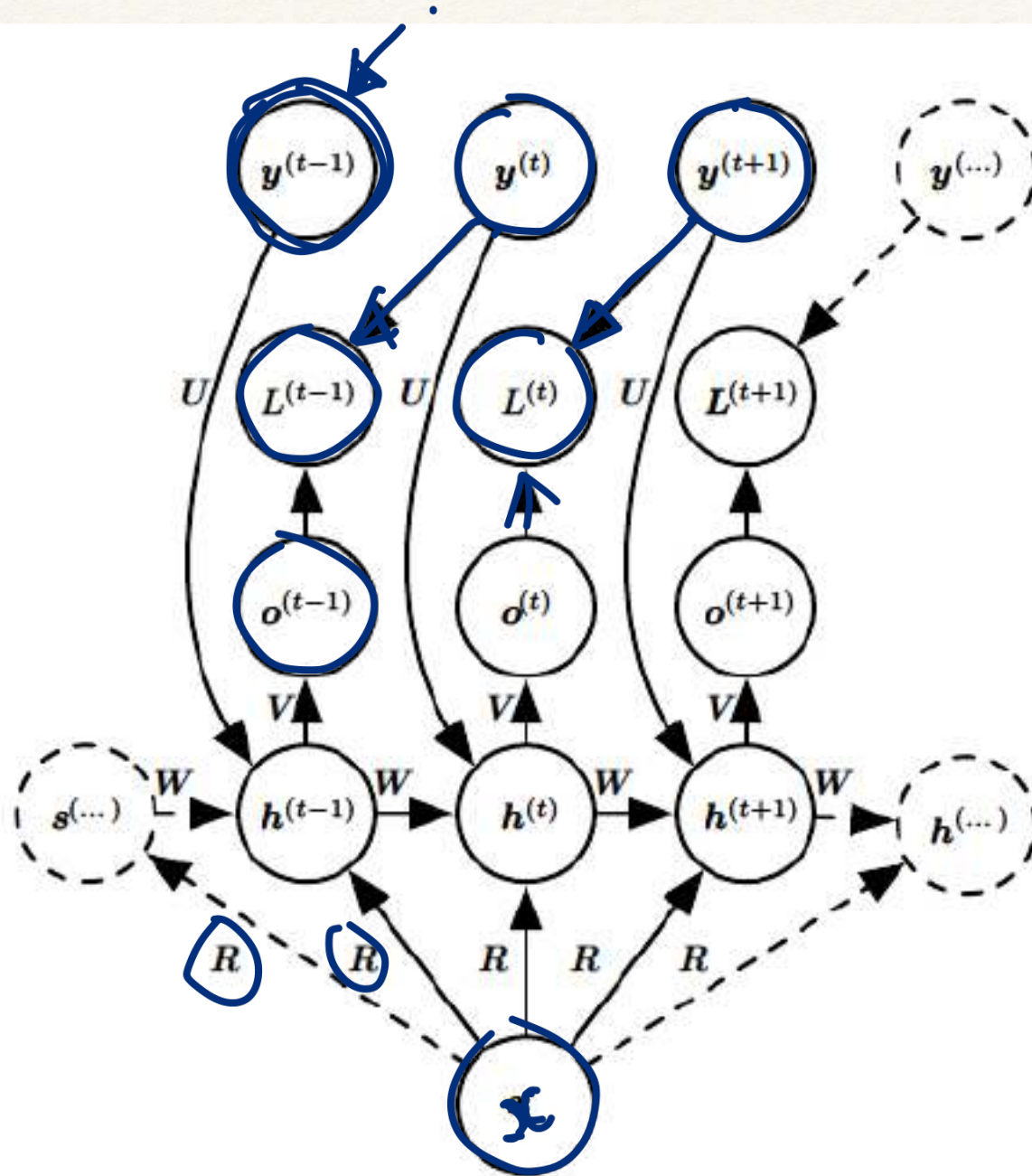
Recurrent Networks



Multiple Input
Single Output

Variable length
inputs.

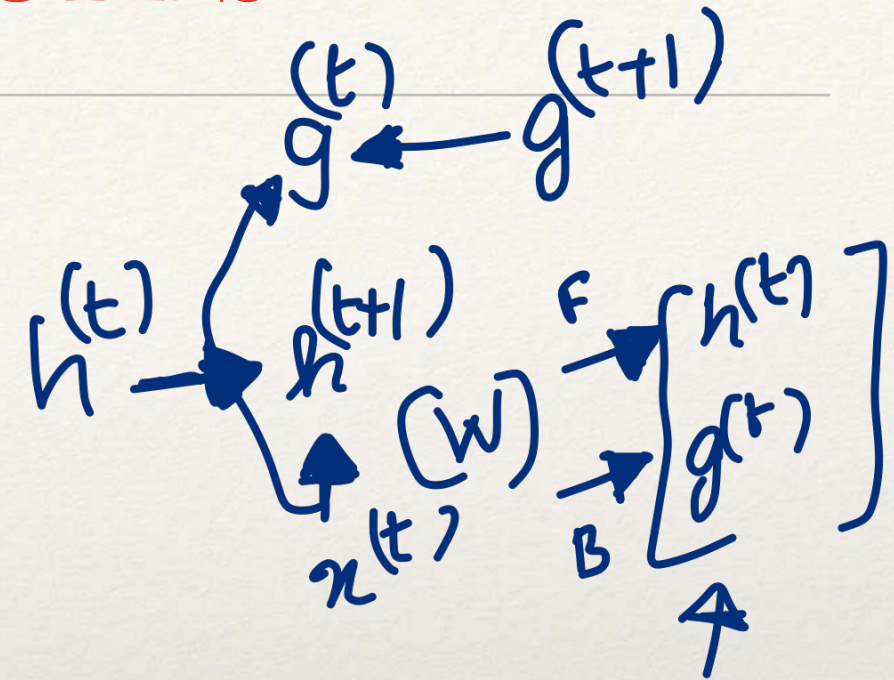
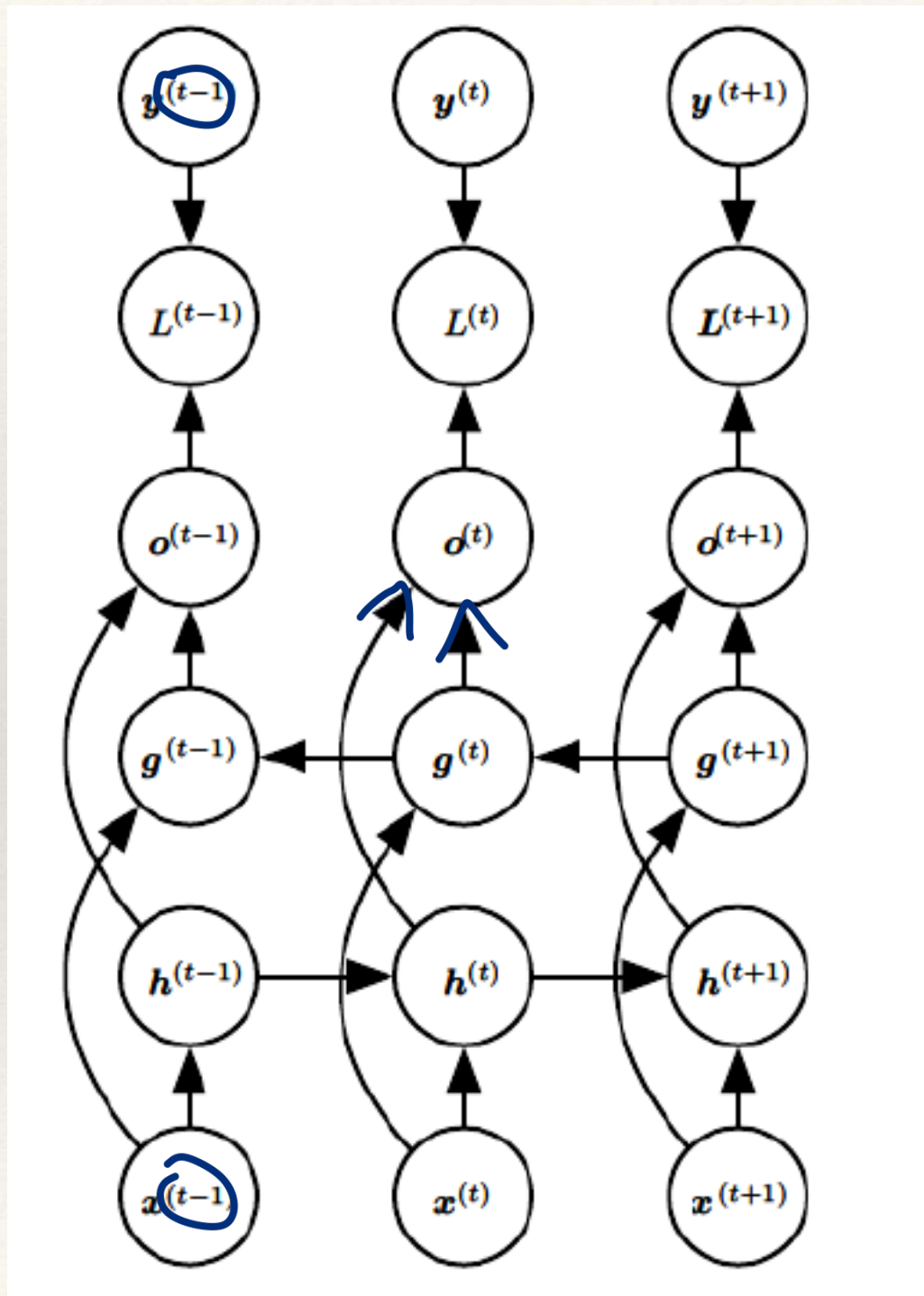
Recurrent Networks



**Single Input
Multiple Output**

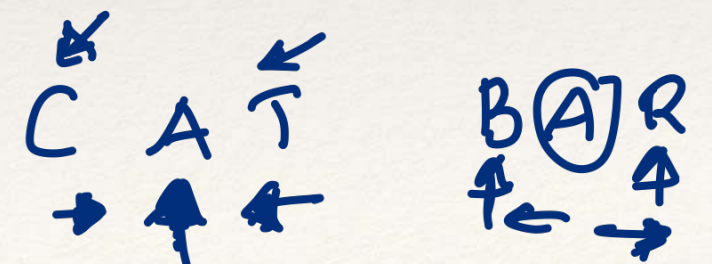
*Autoregressive
Models*

Recurrent Networks

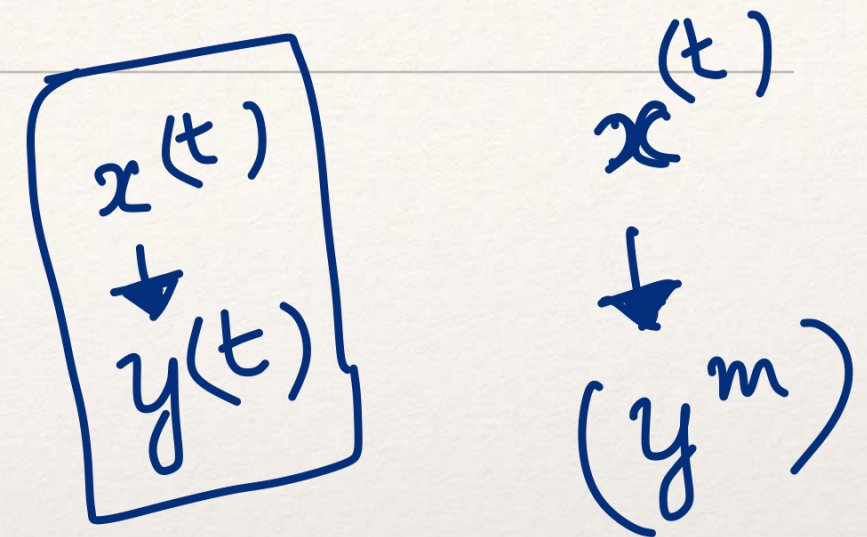
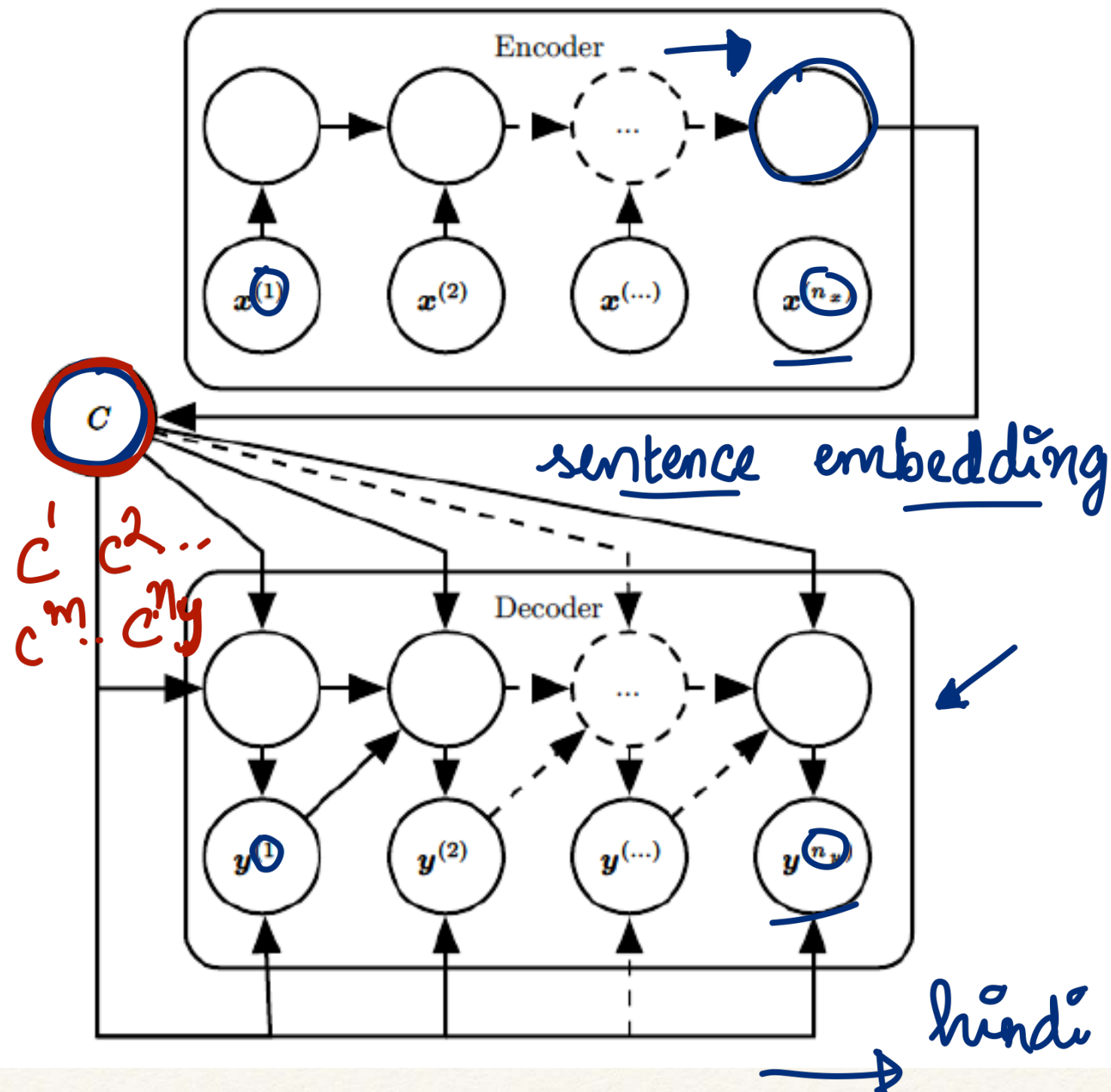


Bi-directional Networks

$[W_1, W_2]$ multiple input
single output



Recurrent Networks



**Sequence to
Sequence
Mapping Networks**

Encoder $\leftarrow$ sequence
to vector

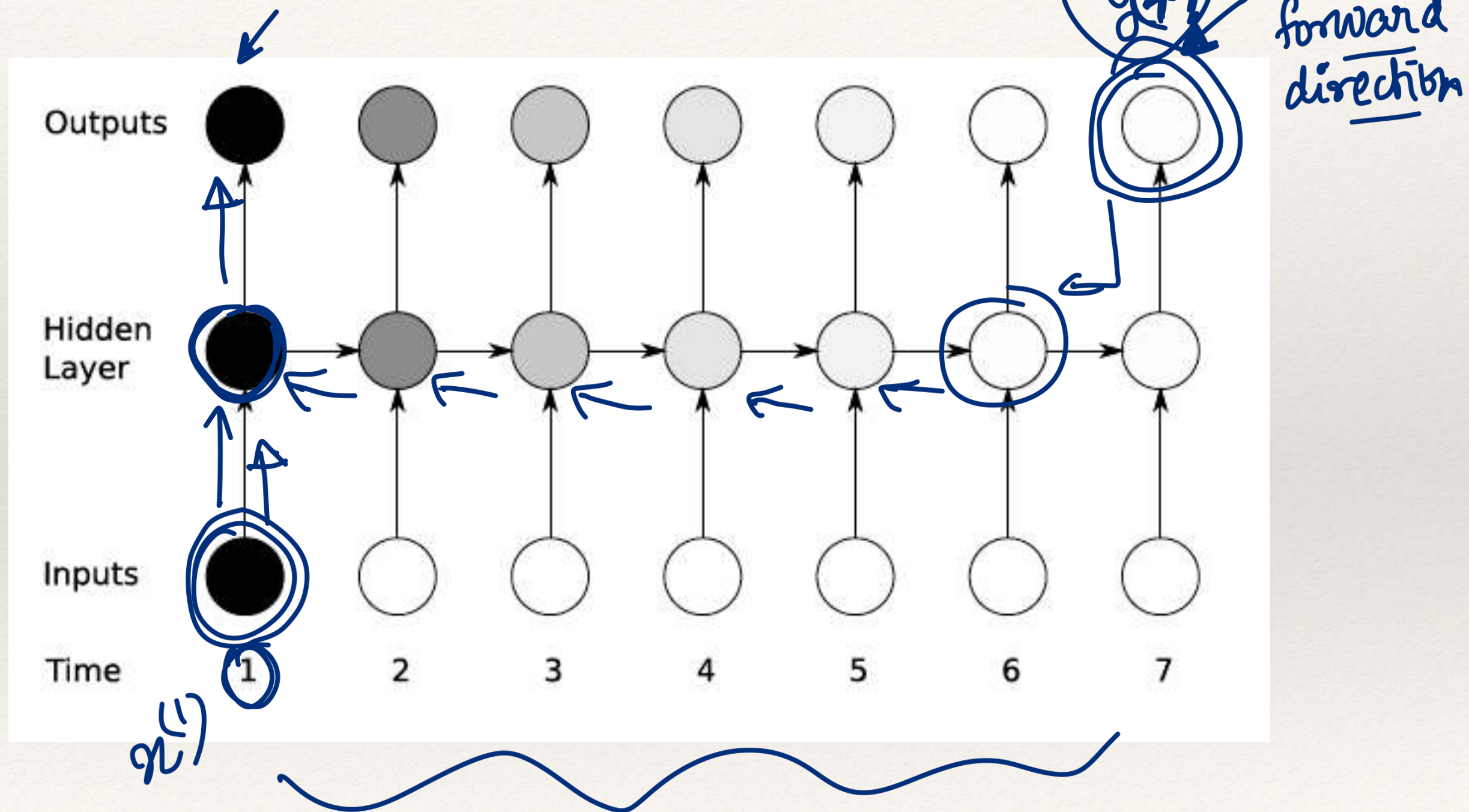
Decoder $\leftarrow$ vector to
a new sequence

$$\textcircled{X} = \{ \underset{\uparrow}{x^{(1)}} \dots \underset{\uparrow}{x^{(T)}} \} \rightarrow \{ \text{Eng Senten} \}$$

$$\textcircled{Y} = \{ y^{(1)} \dots \underset{\downarrow}{y^{(M)}} \} \rightarrow \{ \text{Hindi Sentence} \}$$

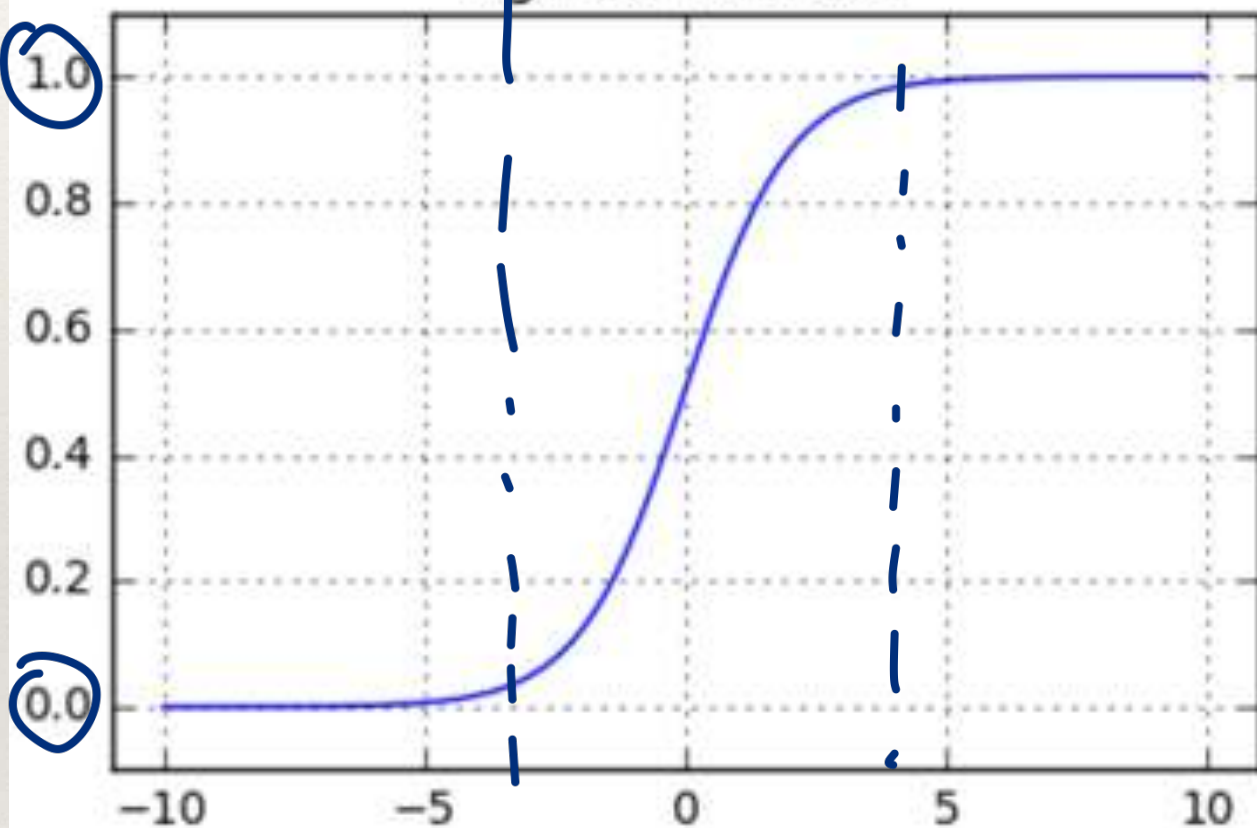


Long-term Dependency Issues

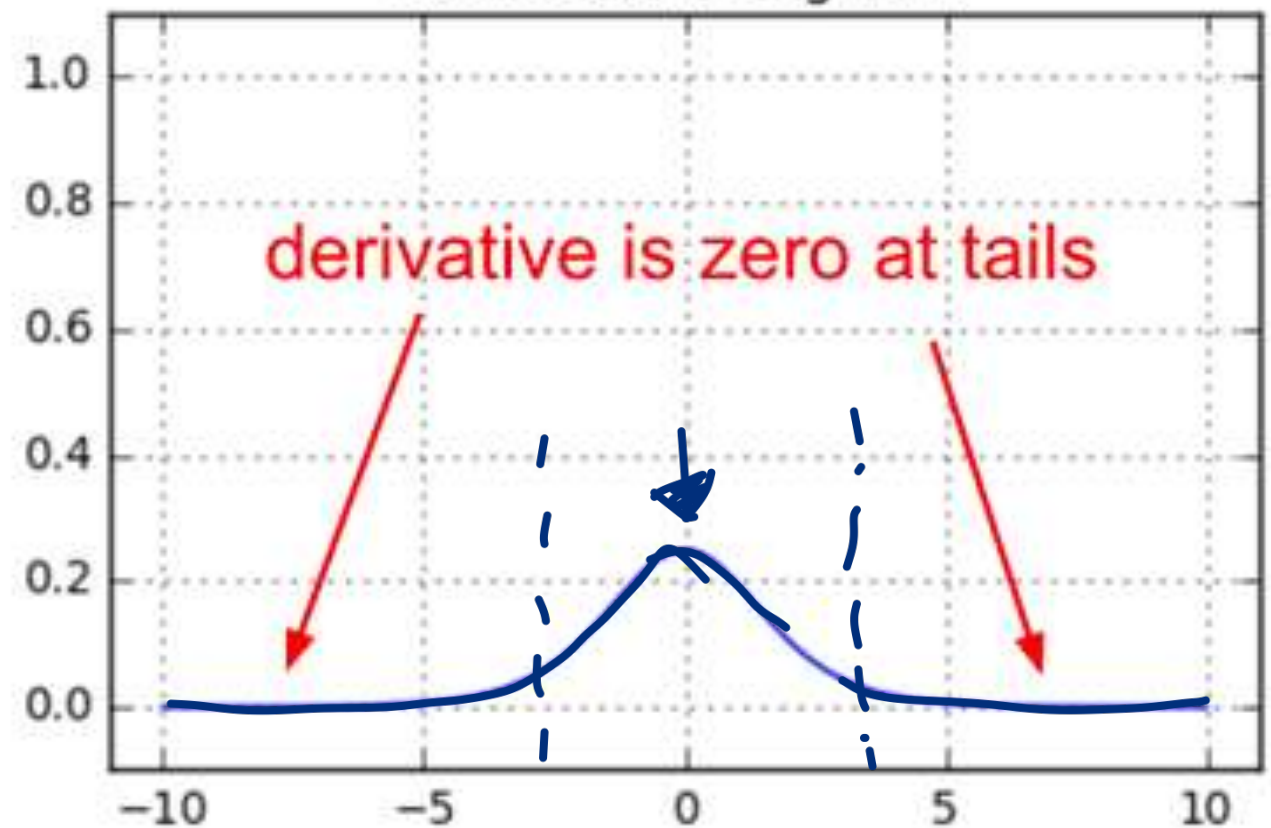


Vanishing/Exploding Gradients

sigmoid function

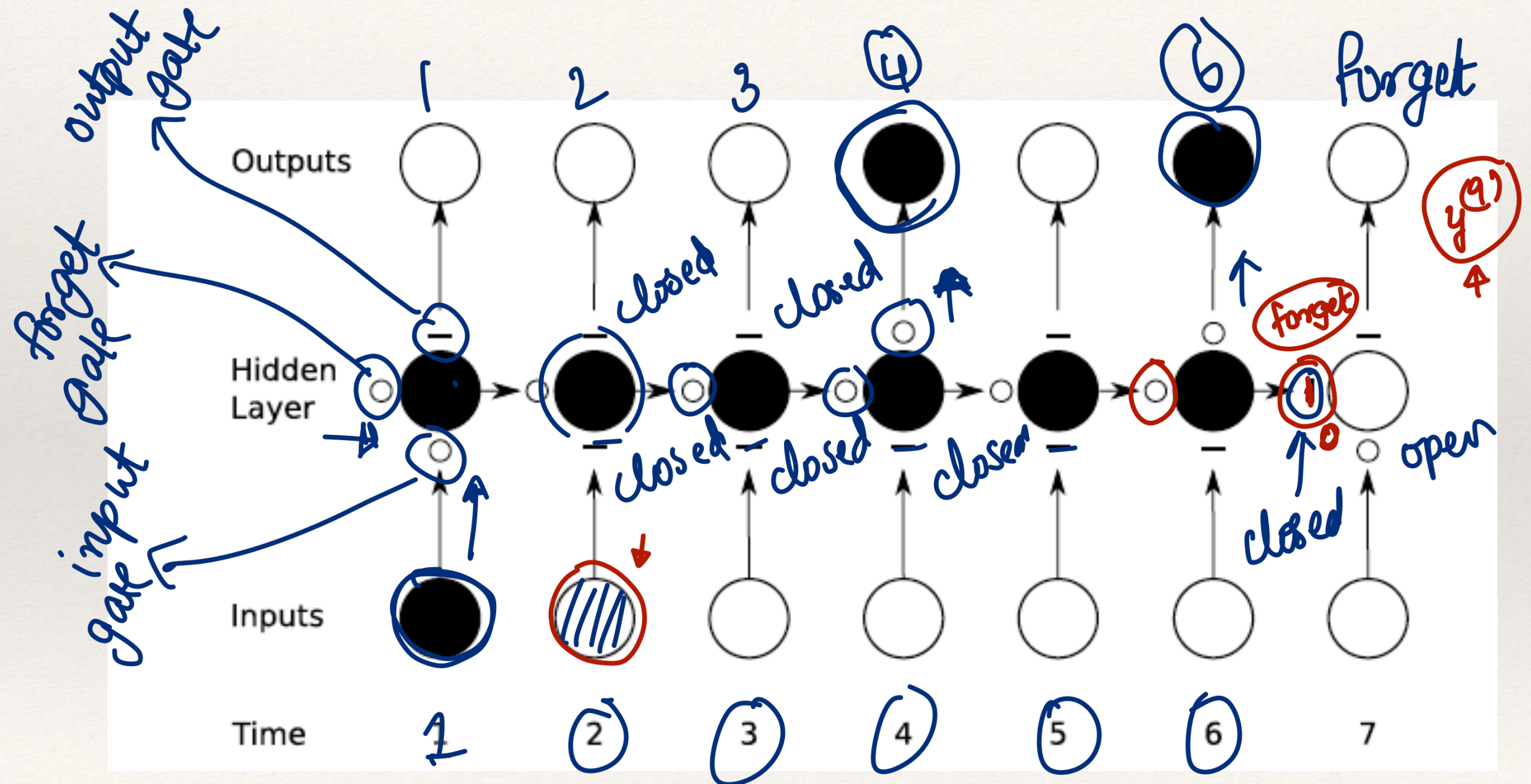


derivative of sigmoid



- ❖ Gradients either vanish or explode
- ❖ Initial frames may not contribute to gradient computations or may contribute too much.

Long-Short Term Memory



LSTM Cell

f - sigmoid function
g, h - tanh function

Input Gate

$$a_l^t = \sum_{i=1}^I w_{il} x_i^t + \sum_{h=1}^H w_{hl} b_h^{t-1} + \sum_{c=1}^C w_{cl} s_c^{t-1}$$
$$b_l^t = f(a_l^t)$$

Forget Gate

$$a_\phi^t = \sum_{i=1}^I w_{i\phi} x_i^t + \sum_{h=1}^H w_{h\phi} b_h^{t-1} + \sum_{c=1}^C w_{c\phi} s_c^{t-1}$$
$$b_\phi^t = f(a_\phi^t)$$

Cell

$$a_c^t = \sum_{i=1}^I w_{ic} x_i^t + \sum_{h=1}^H w_{hc} b_h^{t-1}$$
$$s_c^t = b_\phi^t s_c^{t-1} + b_l^t g(a_c^t)$$

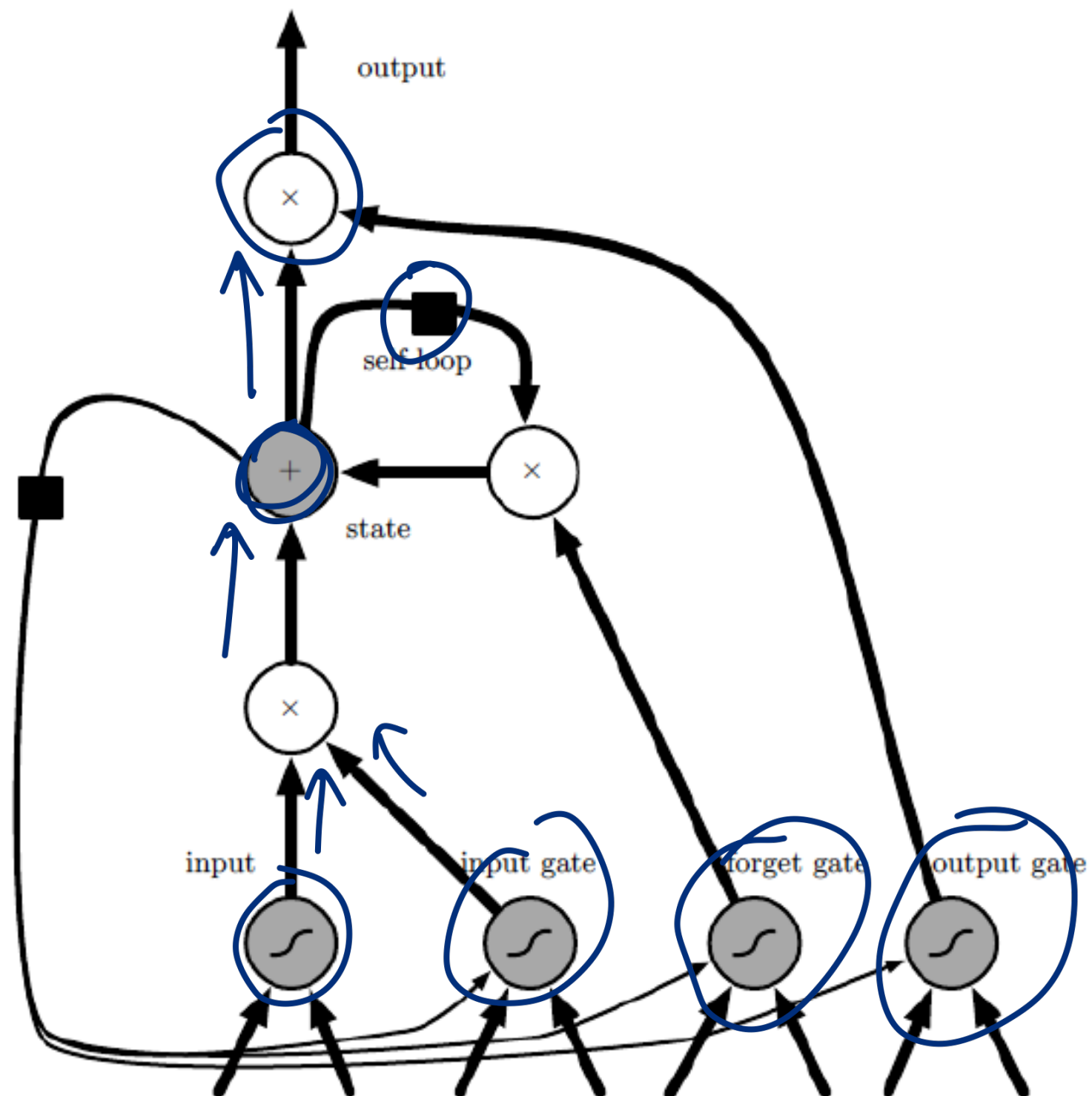
Output Gate

$$a_\omega^t = \sum_{i=1}^I w_{i\omega} x_i^t + \sum_{h=1}^H w_{h\omega} b_h^{t-1} + \sum_{c=1}^C w_{c\omega} s_c^t$$
$$b_\omega^t = f(a_\omega^t)$$

LSTM output

$$b_c^t = b_\omega^t h(s_c^t)$$

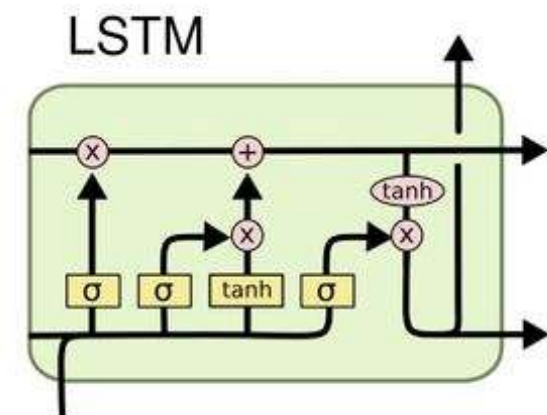
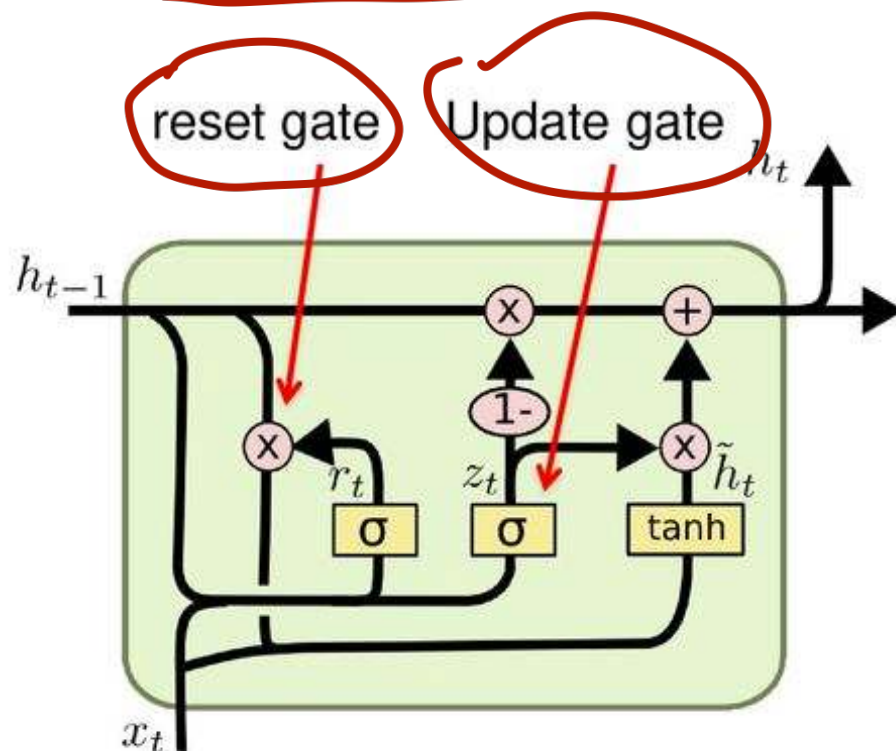
Long Short Term Memory Networks



Gated Recurrent Units (GRU)

GRU – gated recurrent unit

(more compression)



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

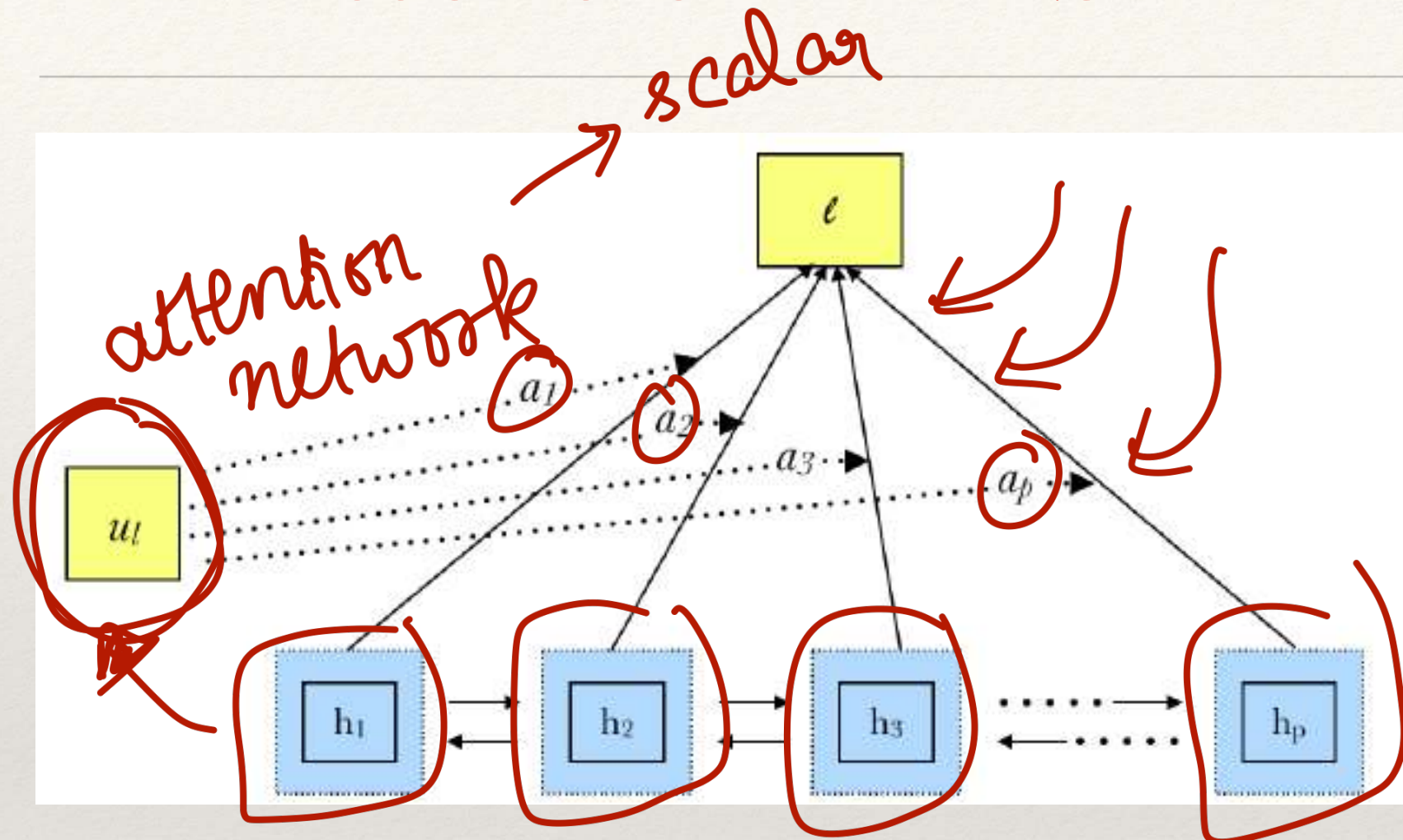
$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

It combines the forget and input into a single update gate. It also merges the cell state and hidden state. This is simpler than LSTM. There are many other variants too.

X,*: element-wise multiply

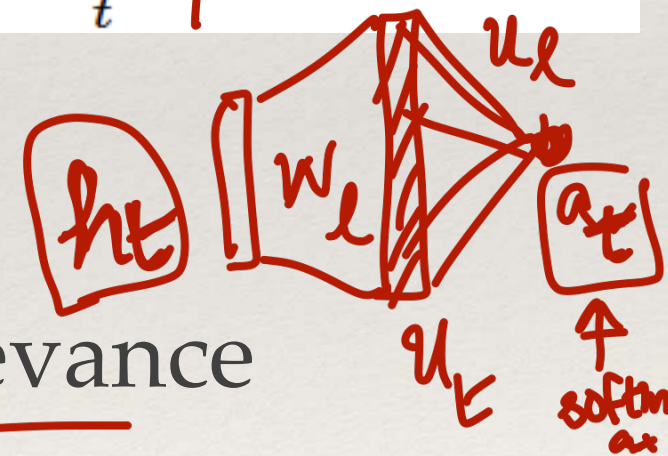
Attention in LSTM Networks



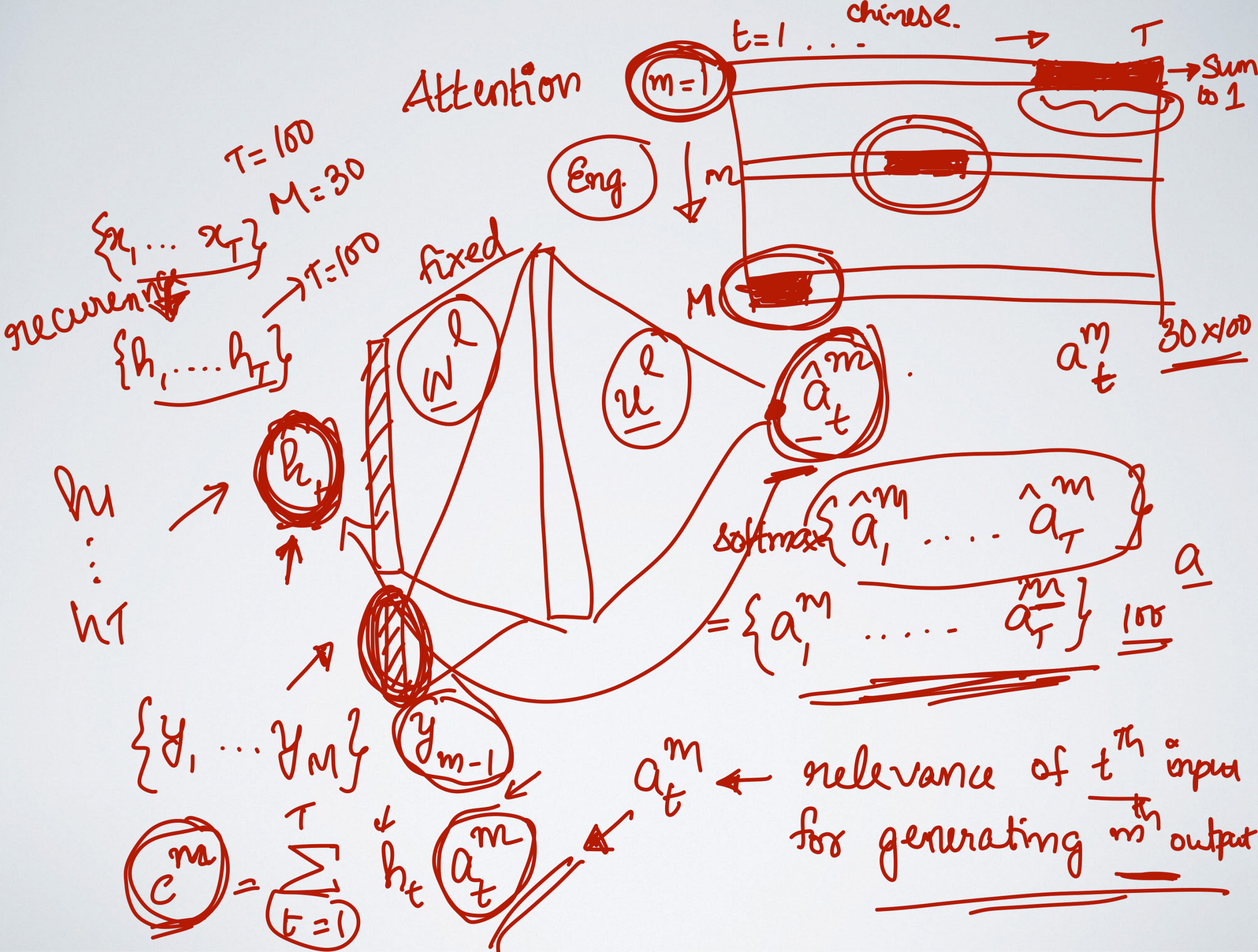
$$\mathbf{u}_t = \tanh(\mathbf{W}_l \mathbf{h}_t + \mathbf{b}_l)$$

$$a_t = \frac{\exp(\mathbf{u}_t^T \mathbf{u}_l)}{\sum_t \exp(\mathbf{u}_t^T \mathbf{u}_l)}$$

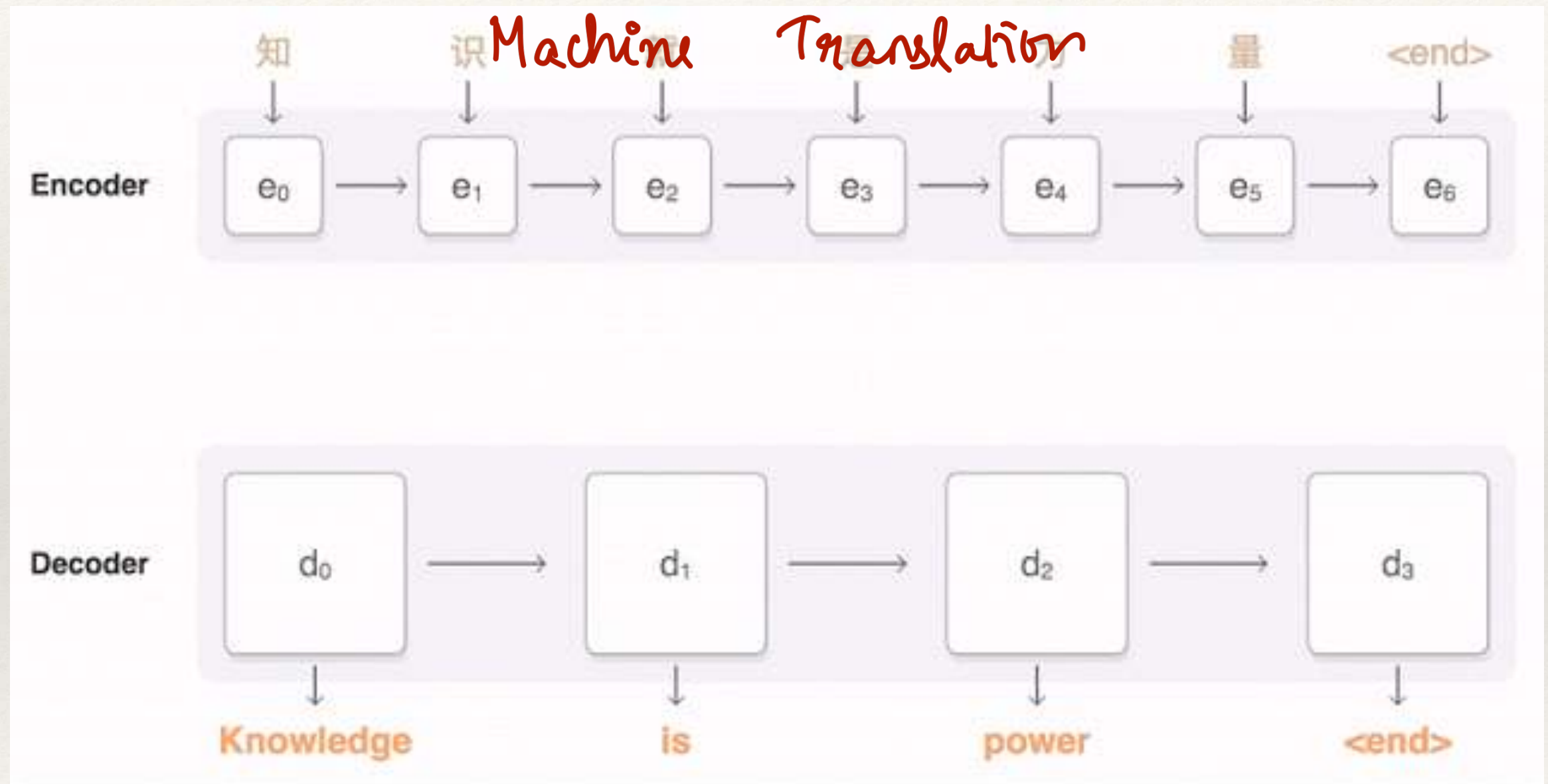
$$\mathbf{l} = \sum_t a_t \mathbf{h}_t$$



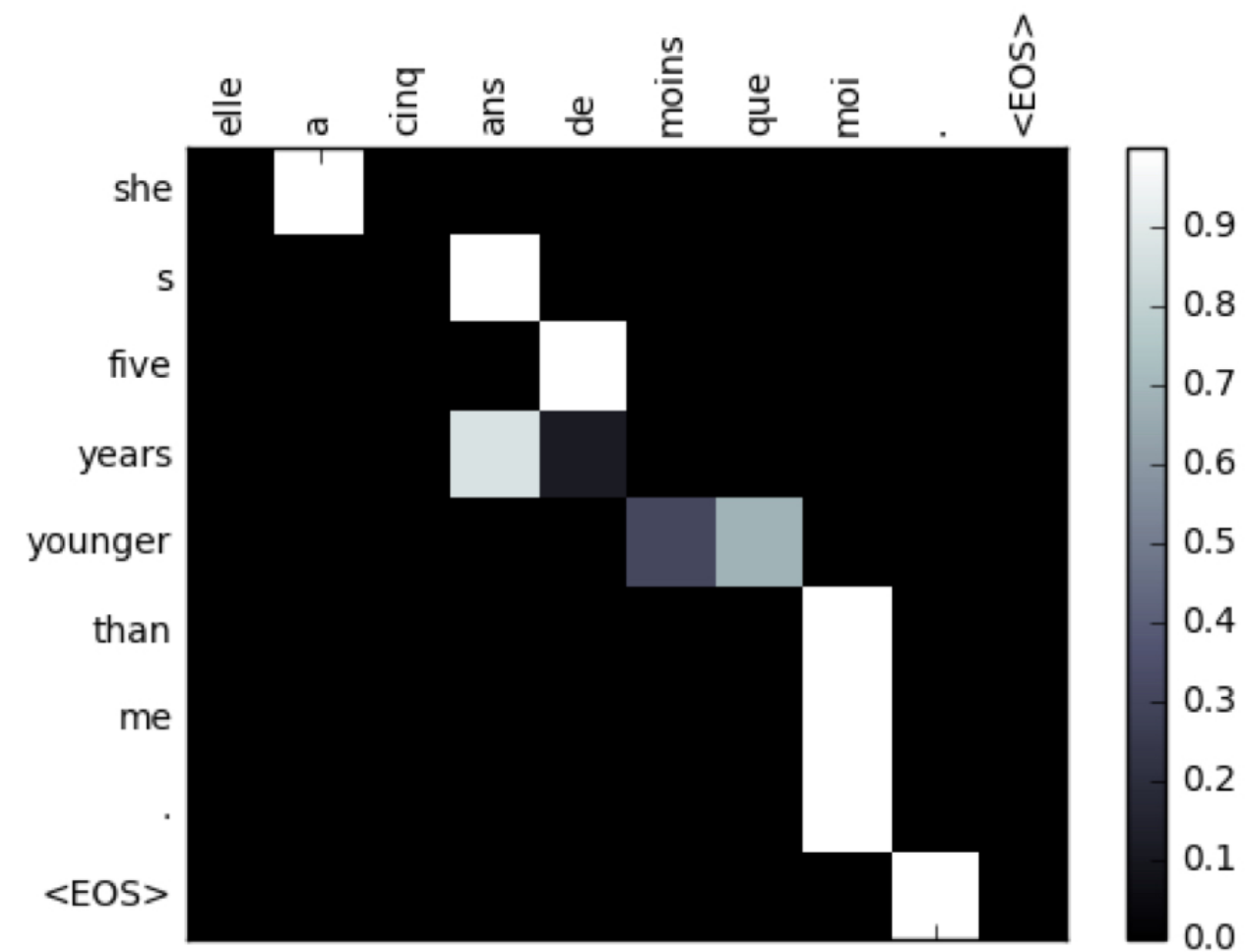
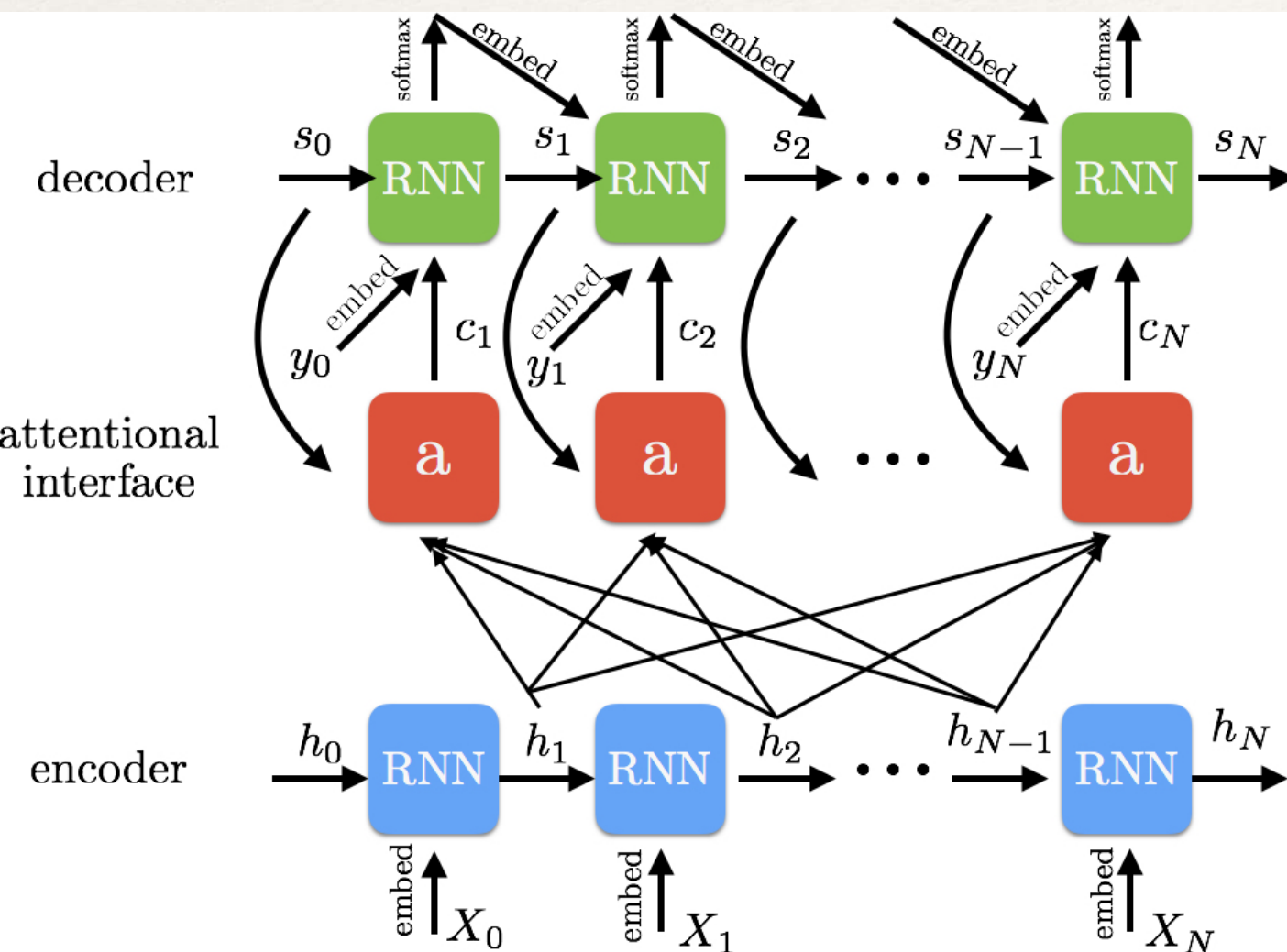
- ❖ Attention allows a mechanism to add relevance
- ❖ Certain regions of the audio have more importance than the rest for the task at hand.



Encoder - Decoder Networks with Attention



Attention Models



Attention – Speech Example

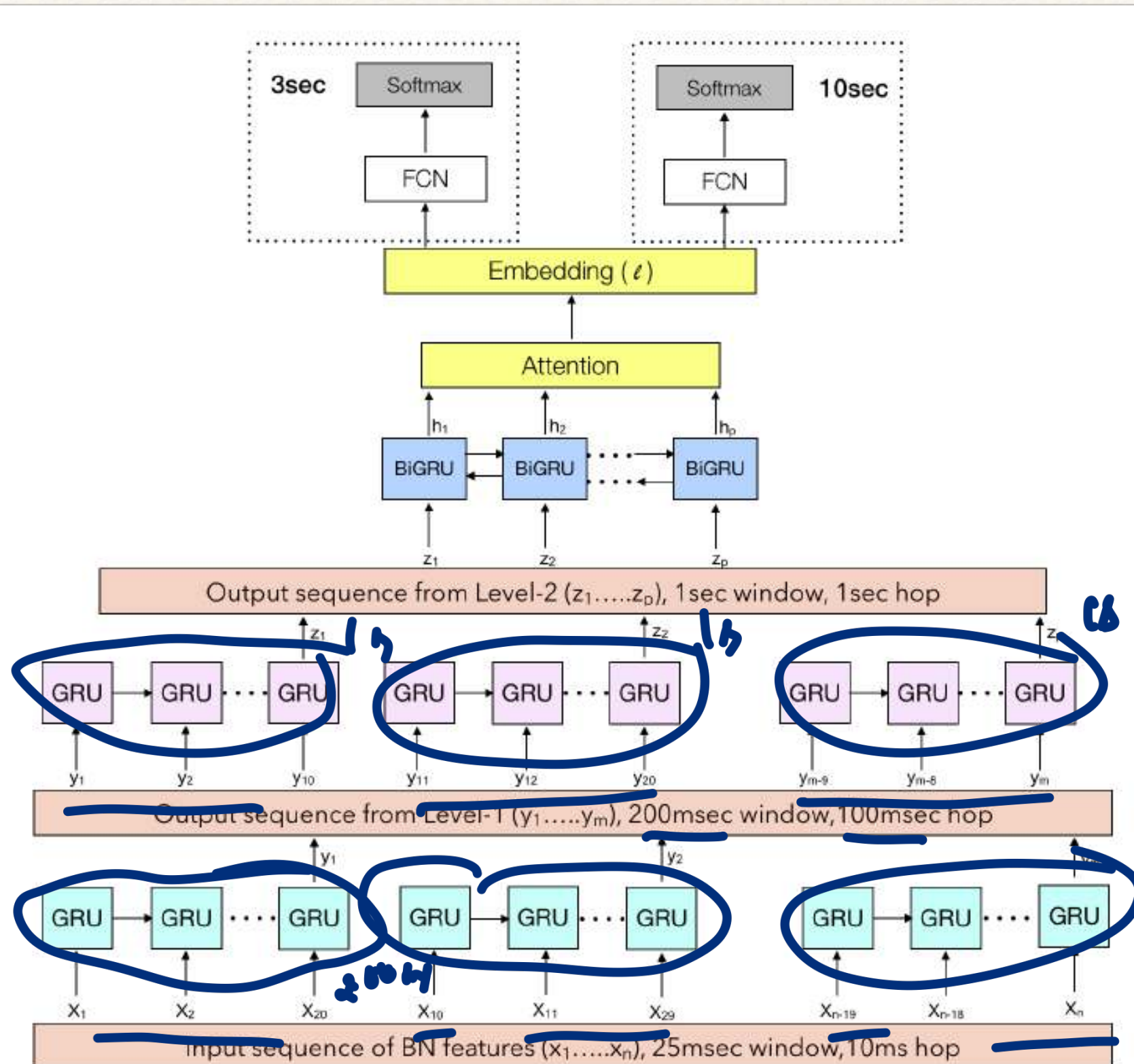
From our lab [part of ICASSP 2019 paper].

Language Recognition Evaluation

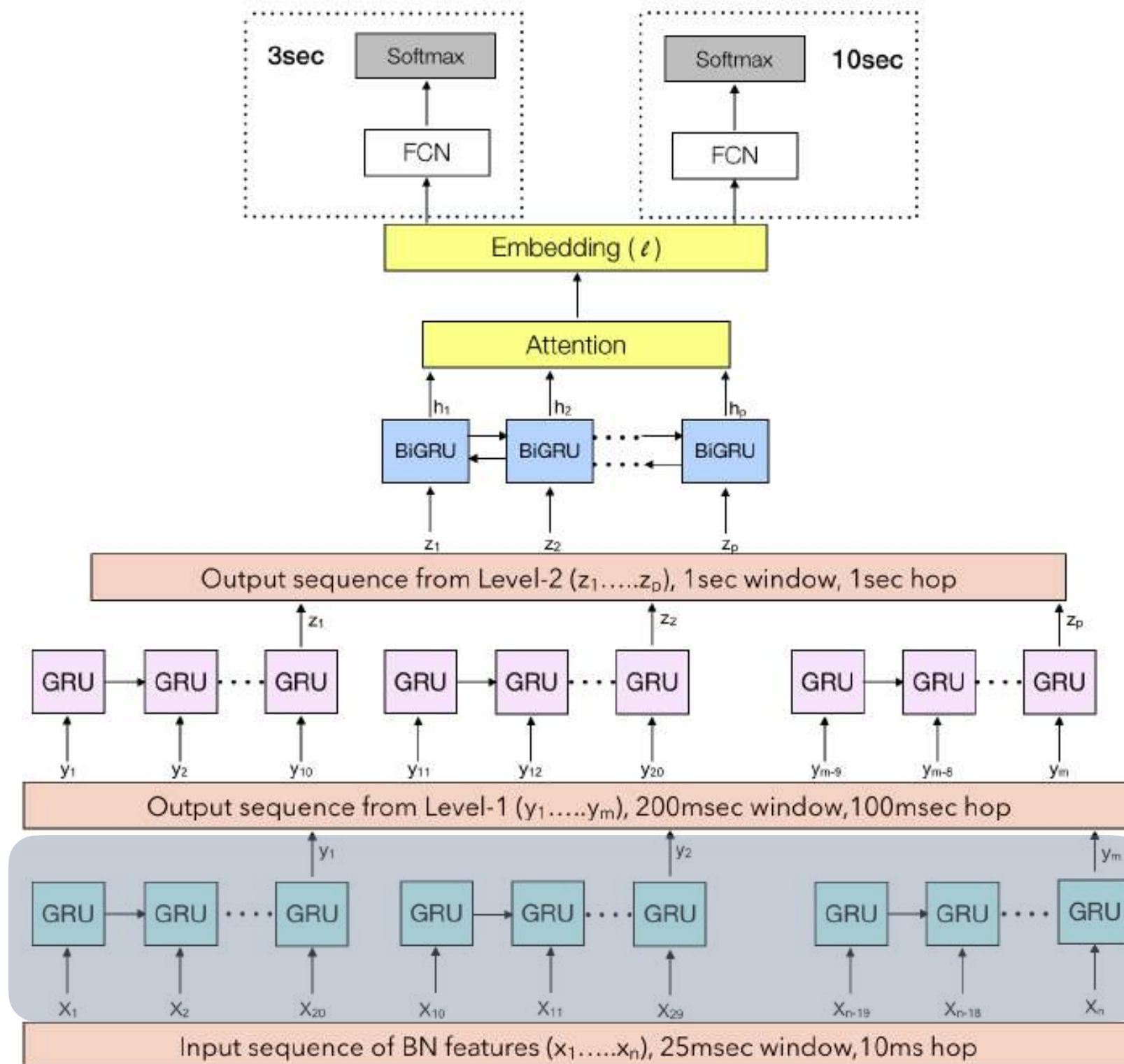
Table 1: LRE17 training set : target languages, language clusters and total number of hours.

Cluster	Target Languages	Hours
Arabic	Egyptian Arabic (ara-arz)	190.9
	Iraqi Arabic (ara-acm)	130.8
	Levantine Arabic (ara-apc)	440.7
	Maghrebi Arabic (ara-ary)	81.8
Chinese	Mandarin (zho-cmn)	379.4
	Min Nan (zho-nan)	13.3
English	British English (eng-gbr)	4.8
	General American English (eng-usg)	327.7
Slavic	Polish (qsl-pol)	59.3
	Russian (qsl-rus)	69.5
Iberian	Caribbean Spanish (spa-car)	166.3
	European Spanish (spa-eur)	24.7
	Latin American Continental Spanish (spa-lac)	175.9
	Brazilian Portuguese (por-brz)	4.1

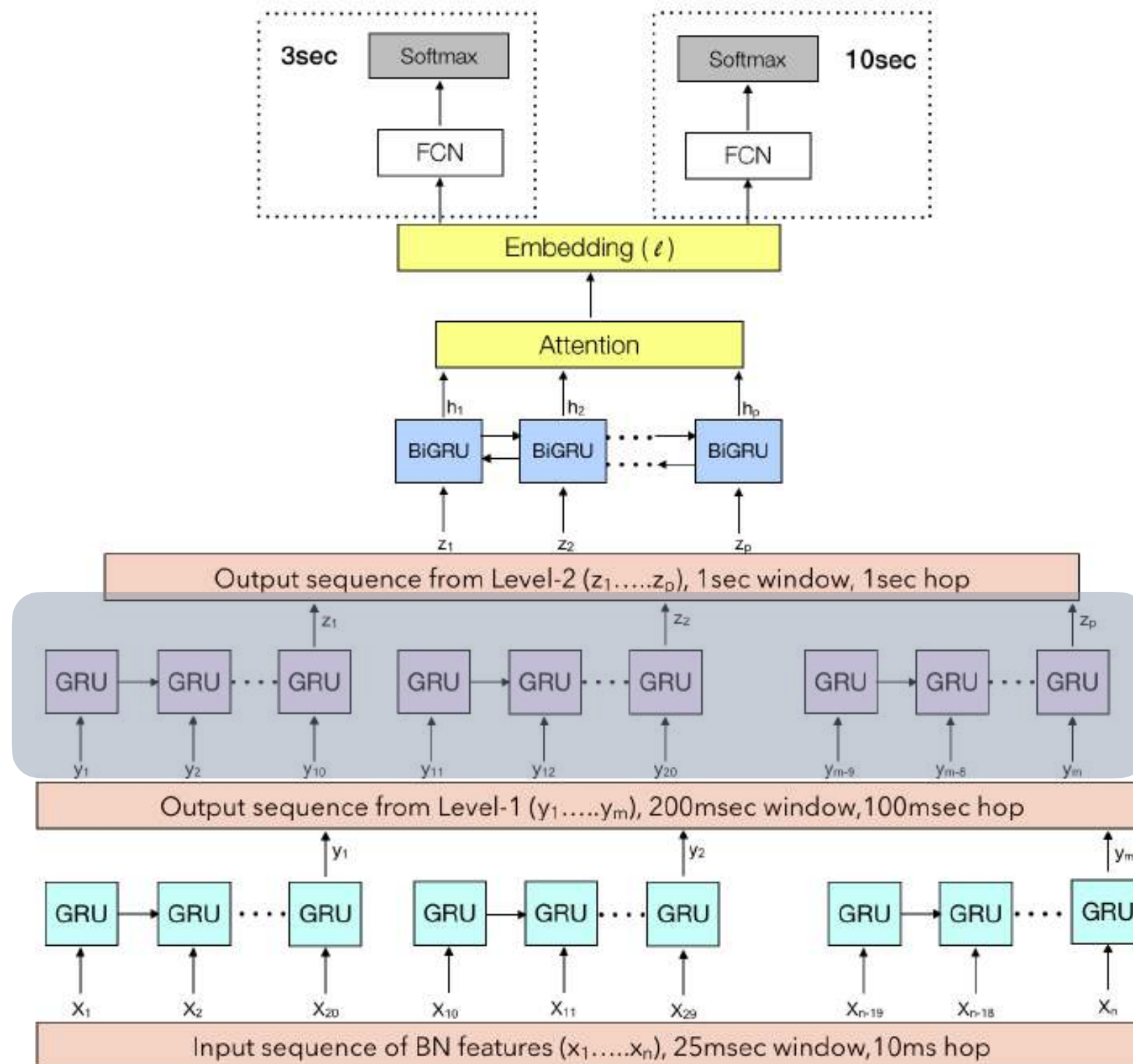
End-to-end model using GRUs and Attention



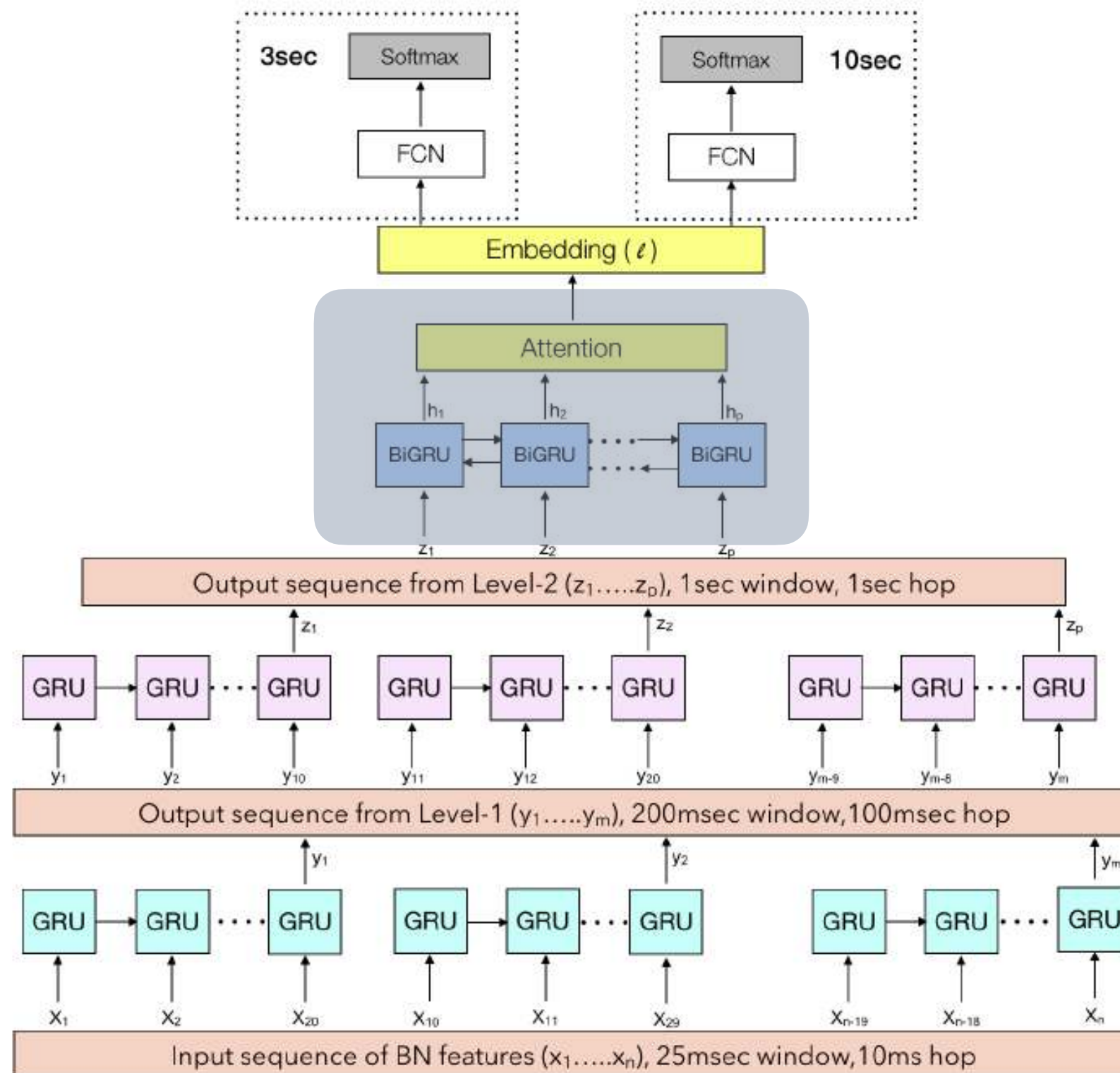
Proposed End-to-End Language Recognition Model



Proposed End-to-End Language Recognition Model



Proposed End-to-End Language Recognition Model



Language Recognition Evaluation

- ❖ State-of-art models use the input sequence directly.
- ❖ We proposed the attention model - Attention weighs the importance of each short-term segment feature for the task.

Attention Weight

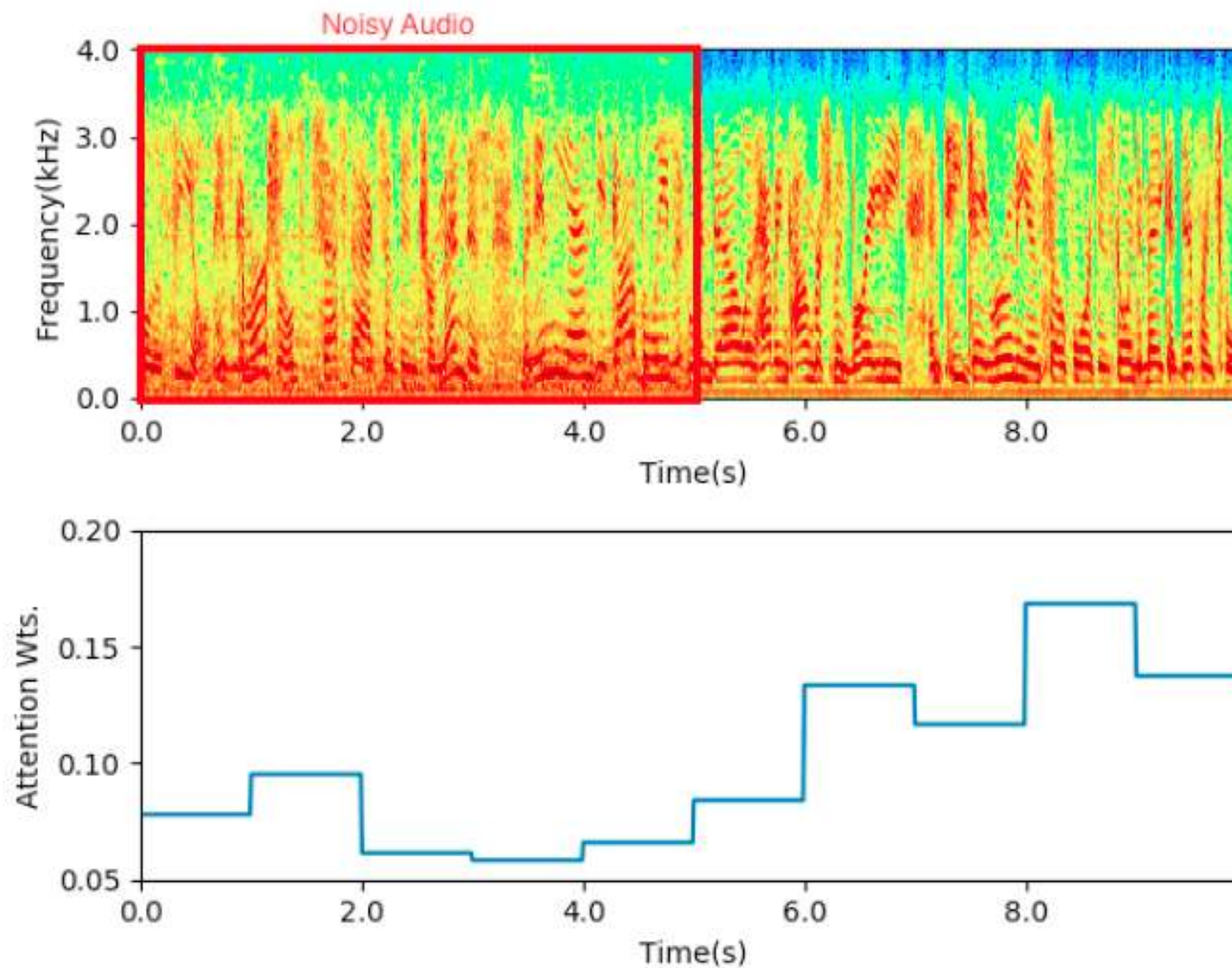
0-3s : O...One muscle at all, it was terrible

3s-4s : ah ah

4s - 9s : I couldn't scream, I couldn't shout, I
couldn't even move my arms up, or my legs

9s -11s : I was trying me hardest, I was really
really panicking.

Language Recognition Evaluation



Language Recognition Evaluation

Table 3. Approximate computational time in seconds for ten 30sec eval files using a single CPU. Machine Specification: 32 CPU, 8 core, 2 thread Intel x86_64 machine with 16 GB Nvidia Quadro P5000 GPU cards.

	ivec. [19]	LSTM [16]	HGRU
CPU	12	51	8
GPU	12	11.5	1.5

Table 4. LID accuracy in % for additional experiments with multiple speakers speaking the same language and the experiments without any SAD information.

Cond.	i-vec. [19]	HGRU
Multi-Speaker	60.6	67.7
Without SAD information	49.7	52.7